VIT®
BHOPAL
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSA4008 - APPLIED MACHINE LEARNING

REG.NO : 23BAI10387

NAME : Devesh Rathore

BRANCH : CSE-AIML

SEMESTER: Fall Semester 2025-26

INDEX

EXP.NO: 01	IMPLEMENT DECISION TREE LEARNING
DATE:09/07/25	

AIM

Implement Decision Tree learning

PROCEDURE

1. **Unsupervised Learning- Reinforcement- theory of learning – feasibility of learning – error and noise – training**
2. **Cvdv**
3. **csfd**

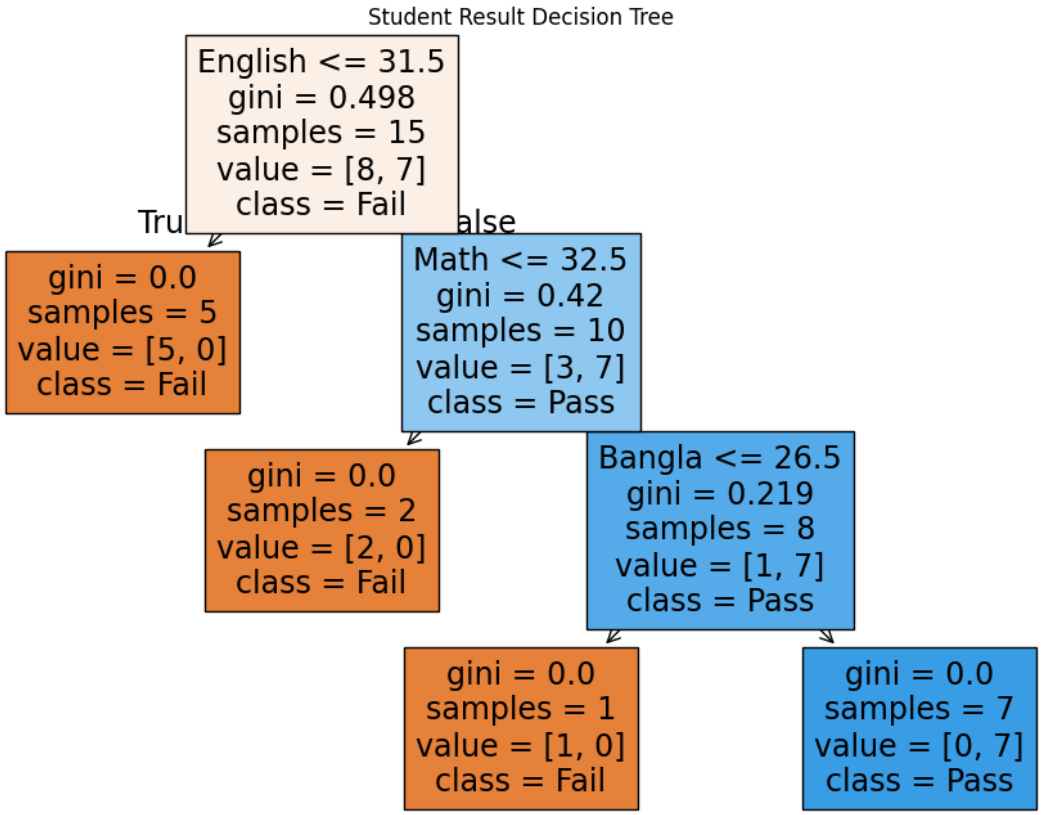
PROGRAM

```python
# Decision Tree for Student Results
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('student_result.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

# Training the Decision Tree model
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier(random_state = 0)
classifier.fit(X, y)

# Predicting a new result (math=65, bangla=45, english=55)
print(classifier.predict([[65, 45, 55]]))

# Visualizing the Decision Tree
from sklearn.tree import plot_tree
plt.figure(figsize=(12, 8))
plot_tree(classifier, feature_names=['Math', 'Bangla', 'English'],
          class_names=['Fail', 'Pass'], filled=True)
plt.title('Student Result Decision Tree')
plt.show()

# Feature importance
feature_importance = pd.DataFrame({
    'Feature': ['Math', 'Bangla', 'English'],
    'Importance': classifier.feature_importances_
}).sort_values('Importance', ascending=False)
print("\nFeature Importance:")
print(feature_importance)
```

INPUT

```
[[80, 70, 60]]

[[20, 30, 25]]

[[33, 33, 33]]
```

OUTPUT



RESULT

```
Feature Importance:
   Feature  Importance
2  English    0.437500
0     Math    0.328125
1   Bangla    0.234375
```

EXP.NO: 02	Implement Logistic Regression
DATE: 16/07/25	

AIM

Implement Logistic Regression

PROCEDURE

Step 1: Data Preparation

The algorithm receives input data in matrix form:

- **X**: Feature matrix (n_samples × n_features)
- **y**: Target vector (n_samples × 1) with binary values (0, 1)

Step 2: Model Initialization

The logistic regression model initializes:

- **Coefficients (β)**: Random small values for each feature
- **Intercept (β_0)**: Initial value (usually 0)
- **Learning rate**: Controls optimization speed

Step 3: Linear Combination Calculation

For each data point, the model computes:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Where z is the linear combination of features and their weights.

Step 4: Sigmoid Function Application

The linear output is transformed using the sigmoid function:

$$P(y=1) = 1 / (1 + e^{(-z)})$$

This maps any real number to a probability between 0 and 1.

Step 5: Cost Function Calculation

The model uses log-likelihood as the cost function:

$$Cost = -[y \cdot log(P) + (1-y) \cdot log(1-P)]$$

This measures the difference between predicted and actual probabilities.

Step 6: Gradient Calculation

The algorithm computes partial derivatives:

$$\partial Cost/\partial \beta = X^T \cdot (P - y) \partial Cost/\partial \beta_0 = \Sigma(P - y)$$

These gradients indicate the direction to update parameters.

Step 7: Parameter Update

Using gradient descent, parameters are updated:

$$\beta = \beta - \alpha \cdot \partial Cost/\partial \beta \beta_0 = \beta_0 - \alpha \cdot \partial Cost/\partial \beta_0$$

Where α is the learning rate.

Step 8: Iteration

Steps 3-7 repeat until:

- Maximum iterations reached
- Cost function converges (change < tolerance)
- Gradients become sufficiently small

Step 9: Final Model

The trained model contains:

- **Optimized coefficients**: Final β values
- **Intercept**: Final β_0 value
- **Decision boundary**: Hyperplane separating classes

Step 10: Prediction Process

For new data points:

1. Calculate linear combination: $z = \beta_0 + \Sigma(\beta_i x_i)$
2. Apply sigmoid function: $P(y=1) = 1/(1+e^{(-z)})$
3. Apply threshold: If $P \geq 0.5 \rightarrow$ class 1, else class 0

PROGRAM

```
# Logistic Regression for Student Results
# Importing the libraries
```

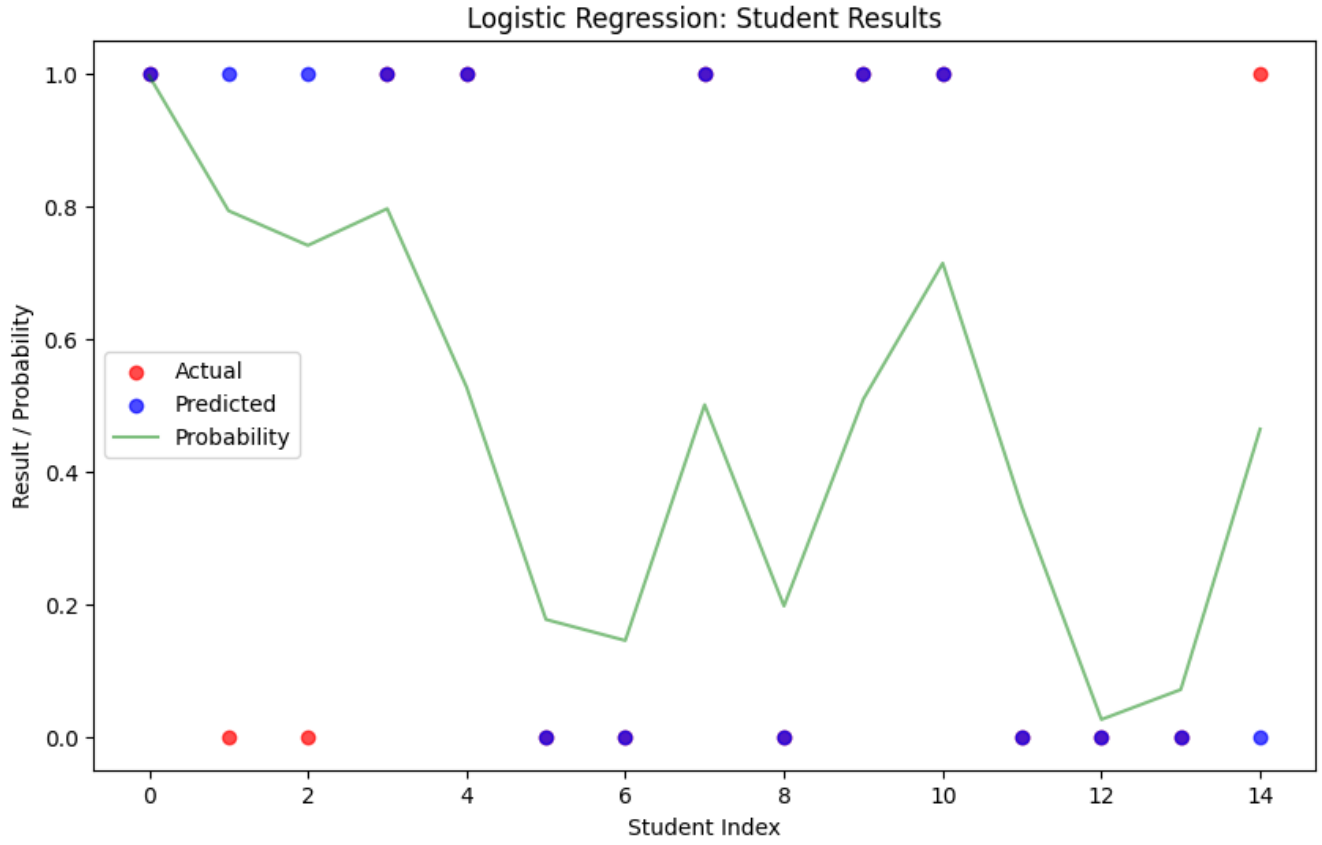
OUTPUT

Probability: [0.05568522 0.94431478]

Model Coefficients:

	Feature	Coefficient
0	Math	0.032050
1	Bangla	-0.033516
2	English	0.106958

Intercept: -3.6269



Model Accuracy: 80.00%

EXP.NO: 03	Implement classification using Multilayer perceptron
DATE:23/07/25	

AIM

Implement classification using Multilayer perceptron

PROCEDURE

Step 1: Environment Setup

python

```
# Install required packages (if not already installed)!pip install pandas numpy matplotlib seaborn scikit-learn# Import librariesimport pandas as pdimport numpy as npimport matplotlib.pyplot as pltimport seaborn as snsfrom sklearn.neural_network import MLPClassifierfrom sklearn.preprocessing import StandardScalerfrom sklearn.model_selection import train_test_splitfrom sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

Step 2: Data Loading and Exploration

Input:

- CSV file with columns: Date, Open, High, Low, Close, Volume, Adj Close
- 365+ rows of historical stock data

Process:

- Load CSV file into pandas DataFrame
- Convert Date column to datetime format
- Sort data chronologically
- Display basic statistics and data info

Expected Output:

```
Original dataset shape: (365, 7)Columns: ['Date', 'Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume']Date range: 2023-11-02 to 2024-11-01
```

Step 3: Feature Engineering

Input: Raw stock data

Process:

- Create technical indicators:
    - Price_Change = Close - Open
    - Price_Change_Pct = (Close - Open) / Open * 100
    - High_Low_Range = High - Low
    - Volume_MA_5 = 5-day moving average of volume
    - Price_MA_5, Price_MA_10 = Moving averages
    - Volatility = (High - Low) / Close * 100
    - RSI = Relative Strength Index
    - Price_Momentum_3, Price_Momentum_5 = Momentum indicators

Expected Output:

```
Dataset shape after preprocessing: (340, 20)New features created: 13 technical indicators
```

Step 4: Target Variable Creation

Input: Preprocessed data with technical indicators

Process:

- Binary target: 1 if next day's close > today's close, 0 otherwise
- Multi-class target based on return thresholds:
    - 0: Strong Down (< -1.5%)
    - 1: Down (-1.5% to -0.5%)
    - 2: Neutral (-0.5% to 0.5%)
    - 3: Up (0.5% to 1.5%)
    - 4: Strong Up (> 1.5%)

Expected Output:

```
Binary Classification Distribution:Class 0 (Down): 165 samplesClass 1 (Up): 175 samplesMulti-class Distribution:Class 0 (Strong Down): 25 samplesClass 1 (Down): 85 samplesClass 2 (Neutral): 130 samplesClass 3 (Up): 80 samplesClass 4 (Strong Up): 20 samples
```

Step 5: Data Preparation

Input: Dataset with features and targets

Process:

- Select relevant features for modeling
- Split data into training (80%) and testing (20%) sets
- Apply StandardScaler to normalize features
- Stratify split to maintain class distribution

Expected Output:

```
Selected features: 13 technical indicatorsFeature matrix shape: (340, 13)Training set shape: (272, 13)Test set shape: (68, 13)
```

Step 6: MLP Model Training

Input: Scaled training data

Process:

- Binary Classification MLP:
    - Architecture: Input → 100 neurons → 50 neurons → Output
    - Activation: ReLU
    - Solver: Adam optimizer
    - Learning rate: Adaptive
- Multi-class Classification MLP:
    - Architecture: Input → 128 → 64 → 32 → Output
    - Same activation and solver settings

Expected Output:

```
Binary Classification Accuracy: 0.6471Multi-class Classification Accuracy: 0.4706Training completed in X iterationsConvergence achieved: True/False
```

Step 7: Model Evaluation

Input: Trained models and test data

Process:

- Generate predictions on test set
- Calculate accuracy, precision, recall, F1-score
- Create confusion matrices
- Plot training loss curves

Expected Output:

Binary Classification Results:

Classification Report: precision recall f1-score support Down 0.62 0.58 0.60 33 Up 0.67 0.71 0.69 35 accuracy 0.65 68 macro avg 0.65 0.65 0.64 68weighted avg 0.65 0.65 0.65 68

Multi-class Classification Results:

Classification Report: precision recall f1-score support Strong Down 0.50 0.40 0.44 5 Down 0.42 0.50 0.46 16 Neutral 0.52 0.58 0.55 26 Up 0.47 0.43 0.45 14 Strong Up 0.33 0.25 0.29 4 accuracy 0.47 68 macro avg 0.45 0.43 0.44 68 weighted avg 0.47 0.47 0.47 68

Step 8: Visualization

Expected Output:

- Confusion matrices (heatmaps)
- Training loss curves
- Feature importance bar chart
- Model performance comparison plots

Step 9: Feature Importance Analysis

Input: Trained model and feature names

Process:

- Calculate permutation importance
- Rank features by importance scores
- Visualize top 10 most important features

Expected Output:

Top 10 Most Important Features: feature importance std0 RSI 0.045122 0.0087431 Volatility 0.038956 0.0072342 Price_Change 0.032847 0.0068913 Price_Momentum_5 0.029384 0.0056724 Volume 0.027193 0.005234...

PROGRAM

```
# Multilayer Perceptron Classification with Comprehensive Analysis
# Designed for Google Colab

import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification, load_digits
from sklearn.model_selection import train_test_split, validation_curve,
learning_curve
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, roc_curve, auc
from sklearn.multiclass import OneVsRestClassifier
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# Set style for better plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

class MLPAnalyzer:
    def __init__(self):
        self.model = None
        self.scaler = StandardScaler()
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None

    def generate_synthetic_data(self, n_samples=2000, n_features=20,
n_classes=3):
        """Generate synthetic classification dataset"""
        X, y = make_classification(
            n_samples=n_samples,
            n_features=n_features,
            n_informative=15,
            n_redundant=5,
            n_classes=n_classes,
            n_clusters_per_class=1,
            random_state=42
        )
        return X, y

    def load_real_data(self):
        """Load real dataset (digits) for comparison"""
        digits = load_digits()
        return digits.data, digits.target

    def prepare_data(self, X, y, test_size=0.2):
        """Prepare and scale the data"""
        self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
            X, y, test_size=test_size, random_state=42, stratify=y
        )
```

```
        # Scale the features
        self.X_train_scaled = self.scaler.fit_transform(self.X_train)
        self.X_test_scaled = self.scaler.transform(self.X_test)

        return self.X_train_scaled, self.X_test_scaled, self.y_train, self.y_test

    def train_mlp(self, hidden_layer_sizes=(100, 50), max_iter=1000):
        """Train the MLP classifier"""
        self.model = MLPClassifier(
            hidden_layer_sizes=hidden_layer_sizes,
            activation='relu',
            solver='adam',
            alpha=0.0001,
            batch_size='auto',
            learning_rate='constant',
            learning_rate_init=0.001,
            max_iter=max_iter,
            random_state=42,
            early_stopping=True,
            validation_fraction=0.1,
            n_iter_no_change=10
        )

        self.model.fit(self.X_train_scaled, self.y_train)
        return self.model

    def plot_data_distribution(self, X, y):
        """Plot data distribution and class balance"""
        fig, axes = plt.subplots(1, 3, figsize=(15, 5))

        # Class distribution
        unique, counts = np.unique(y, return_counts=True)
        axes[0].bar(unique, counts, alpha=0.7, color='skyblue')
        axes[0].set_title('Class Distribution')
        axes[0].set_xlabel('Class')
        axes[0].set_ylabel('Count')

        # Feature correlation heatmap (first 10 features if too many)
        n_features_to_show = min(10, X.shape[1])
        corr_matrix = np.corrcoef(X[:, :n_features_to_show].T)
        im = axes[1].imshow(corr_matrix, cmap='coolwarm', aspect='auto')
        axes[1].set_title(f'Feature Correlation (First {n_features_to_show}
features)')
        plt.colorbar(im, ax=axes[1])

        # PCA visualization for 2D projection
        from sklearn.decomposition import PCA
        pca = PCA(n_components=2)
        X_pca = pca.fit_transform(X)

        scatter = axes[2].scatter(X_pca[:, 0], X_pca[:, 1], c=y, alpha=0.6,
cmap='viridis')
```

```
        axes[2].set_title('PCA Visualization (2D projection)')
        axes[2].set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%}
variance)')
        axes[2].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%}
variance)')
        plt.colorbar(scatter, ax=axes[2])

        plt.tight_layout()
        plt.show()

    def plot_training_history(self):
        """Plot training loss curve"""
        if hasattr(self.model, 'loss_curve_'):
            plt.figure(figsize=(10, 6))
            plt.plot(self.model.loss_curve_, 'b-', label='Training Loss', linewidth=2)
            if hasattr(self.model, 'validation_scores_'):
                plt.plot(self.model.validation_scores_, 'r-', label='Validation Score',
linewidth=2)
            plt.title('Training History')
            plt.xlabel('Iterations')
            plt.ylabel('Loss / Score')
            plt.legend()
            plt.grid(True, alpha=0.3)
            plt.show()

    def plot_confusion_matrix(self, y_true, y_pred):
        """Plot confusion matrix"""
        cm = confusion_matrix(y_true, y_pred)
        plt.figure(figsize=(8, 6))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar_kws={'label':
'Count'})
        plt.title('Confusion Matrix')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.show()
        return cm

    def plot_roc_curves(self, X_test, y_test):
        """Plot ROC curves for multiclass classification"""
        y_score = self.model.predict_proba(X_test)
        n_classes = len(np.unique(y_test))

        # Binarize the output
        from sklearn.preprocessing import label_binarize
        y_test_bin = label_binarize(y_test, classes=range(n_classes))

        plt.figure(figsize=(10, 8))
        colors = plt.cm.Set1(np.linspace(0, 1, n_classes))

        for i, color in zip(range(n_classes), colors):
            fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
            roc_auc = auc(fpr, tpr)
```

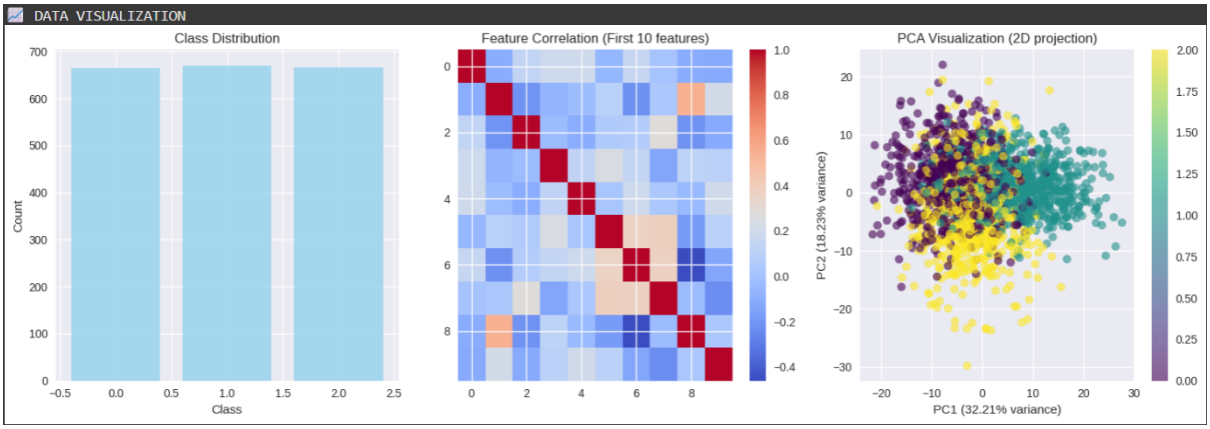
```
            plt.plot(fpr, tpr, color=color, lw=2,
                     label=f'Class {i} (AUC = {roc_auc:.2f})')

        plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Random Classifier')
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('ROC Curves for Multiclass Classification')
        plt.legend(loc="lower right")
        plt.grid(True, alpha=0.3)
        plt.show()

    def plot_learning_curves(self, X, y):
        """Plot learning curves to analyze bias/variance"""
        train_sizes, train_scores, val_scores = learning_curve(
            self.model, X, y, cv=5, n_jobs=-1,
            train_sizes=np.linspace(0.1, 1.0, 10),
            scoring='accuracy'
        )

        plt.figure(figsize=(10, 6))
        plt.fill_between(train_sizes, train_scores.mean(axis=1) -
train_scores.std(axis=1),
                         train_scores.mean(axis=1) + train_scores.std(axis=1),
alpha=0.1, color="blue")
        plt.fill_between(train_sizes, val_scores.mean(axis=1) -
val_scores.std(axis=1),
                         val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=0.1,
color="red")

        plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color="blue",
label="Training Score")
        plt.plot(train_sizes, val_scores.mean(axis=1), 'o-', color="red",
label="Cross-Validation Score")

        plt.title('Learning Curves')
        plt.xlabel('Training Set Size')
        plt.ylabel('Accuracy Score')
        plt.legend(loc="best")
        plt.grid(True, alpha=0.3)
        plt.show()

    def plot_validation_curves(self, X, y):
        """Plot validation curves for hyperparameter tuning"""
        # Analyze alpha (regularization parameter)
        alpha_range = np.logspace(-5, 1, 7)
        train_scores, val_scores = validation_curve(
            MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000,
random_state=42),
            X, y, param_name='alpha', param_range=alpha_range,
            cv=5, scoring='accuracy', n_jobs=-1
```

```
    )

    plt.figure(figsize=(12, 5))

    # Alpha validation curve
    plt.subplot(1, 2, 1)
    plt.semilogx(alpha_range, train_scores.mean(axis=1), 'o-', color="blue",
label="Training Score")
    plt.semilogx(alpha_range, val_scores.mean(axis=1), 'o-', color="red",
label="Cross-Validation Score")
    plt.fill_between(alpha_range, train_scores.mean(axis=1) -
train_scores.std(axis=1),
                     train_scores.mean(axis=1) + train_scores.std(axis=1),
alpha=0.1, color="blue")
    plt.fill_between(alpha_range, val_scores.mean(axis=1) -
val_scores.std(axis=1),
                     val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=0.1,
color="red")
    plt.title('Validation Curve (Alpha)')
    plt.xlabel('Alpha (Regularization)')
    plt.ylabel('Accuracy Score')
    plt.legend(loc="best")
    plt.grid(True, alpha=0.3)

    # Hidden layer size validation curve
    hidden_sizes = [(50,), (100,), (100, 50), (100, 100), (150, 100), (200,
100)]
    train_scores_h, val_scores_h = validation_curve(
        MLPClassifier(alpha=0.0001, max_iter=1000, random_state=42),
        X, y, param_name='hidden_layer_sizes', param_range=hidden_sizes,
        cv=3, scoring='accuracy', n_jobs=-1
    )

    plt.subplot(1, 2, 2)
    x_pos = range(len(hidden_sizes))
    plt.plot(x_pos, train_scores_h.mean(axis=1), 'o-', color="blue",
label="Training Score")
    plt.plot(x_pos, val_scores_h.mean(axis=1), 'o-', color="red",
label="Cross-Validation Score")
    plt.fill_between(x_pos, train_scores_h.mean(axis=1) -
train_scores_h.std(axis=1),
                     train_scores_h.mean(axis=1) + train_scores_h.std(axis=1),
alpha=0.1, color="blue")
    plt.fill_between(x_pos, val_scores_h.mean(axis=1) -
val_scores_h.std(axis=1),
                     val_scores_h.mean(axis=1) + val_scores_h.std(axis=1),
alpha=0.1, color="red")
    plt.title('Validation Curve (Hidden Layer Sizes)')
    plt.xlabel('Architecture')
    plt.ylabel('Accuracy Score')
    plt.xticks(x_pos, [str(h) for h in hidden_sizes], rotation=45)
    plt.legend(loc="best")
```

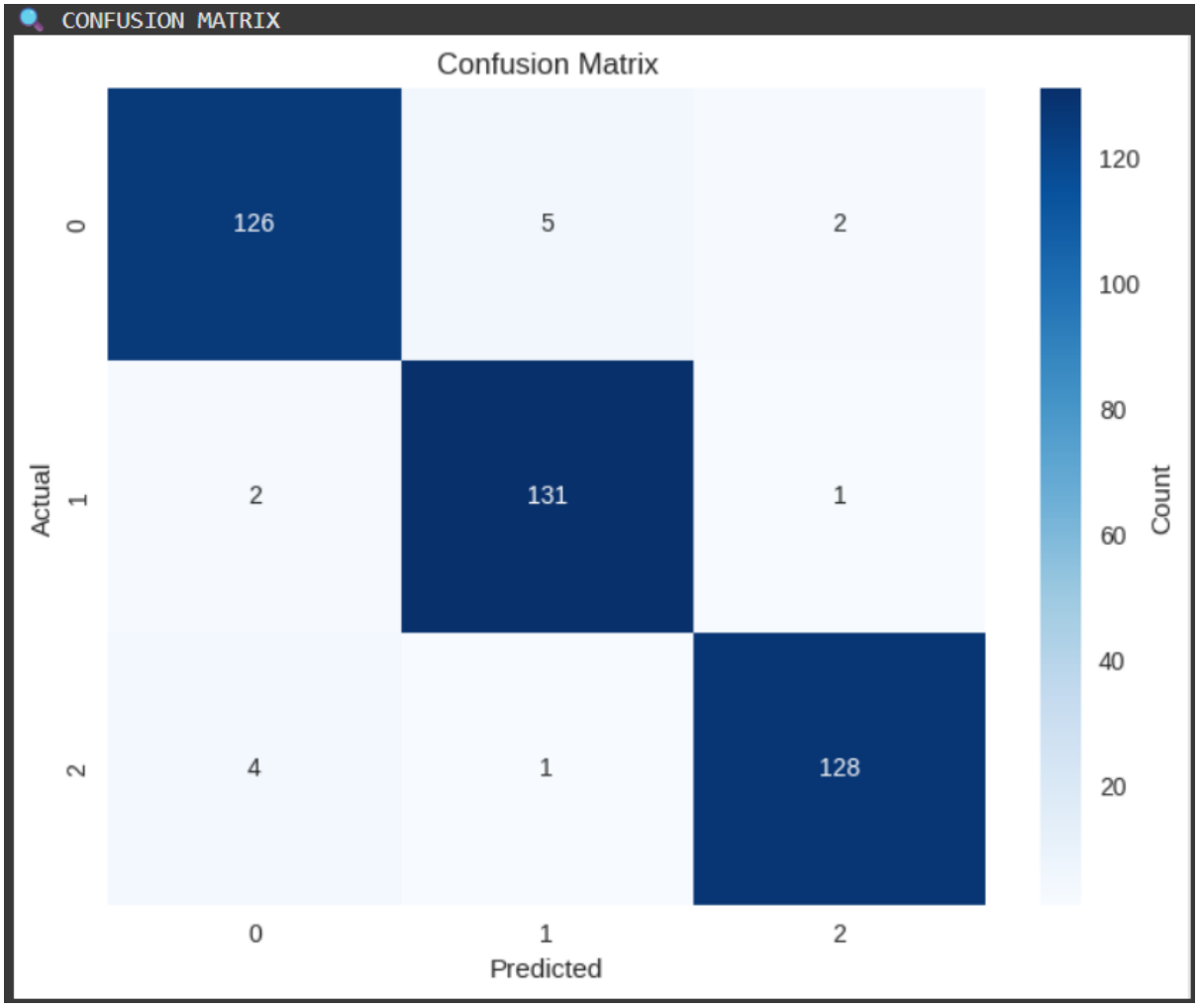
```
        plt.grid(True, alpha=0.3)

        plt.tight_layout()
        plt.show()

    def evaluate_model(self):
        """Comprehensive model evaluation"""
        # Predictions
        y_pred = self.model.predict(self.X_test_scaled)
        y_pred_proba = self.model.predict_proba(self.X_test_scaled)

        # Print classification report
        print("=" * 60)
        print("CLASSIFICATION REPORT")
        print("=" * 60)
        print(classification_report(self.y_test, y_pred))

        # Accuracy
        train_accuracy = accuracy_score(self.y_train,
self.model.predict(self.X_train_scaled))
        test_accuracy = accuracy_score(self.y_test, y_pred)

        print(f"\nAccuracy Scores:")
        print(f"Training Accuracy: {train_accuracy:.4f}")
        print(f"Test Accuracy: {test_accuracy:.4f}")
        print(f"Overfitting Gap: {train_accuracy - test_accuracy:.4f}")

        return y_pred, y_pred_proba

def main():
    """Main execution function"""
    analyzer = MLPAnalyzer()

    print("🧠 MULTILAYER PERCEPTRON CLASSIFICATION ANALYSIS")
    print("=" * 60)

    # Generate and analyze synthetic data
    print("\n📊 GENERATING SYNTHETIC DATASET")
    X_synthetic, y_synthetic = analyzer.generate_synthetic_data()
    print(f"Dataset shape: {X_synthetic.shape}")
    print(f"Number of classes: {len(np.unique(y_synthetic))}")

    # Plot data distribution
    print("\n📈 DATA VISUALIZATION")
    analyzer.plot_data_distribution(X_synthetic, y_synthetic)

    # Prepare data
    print("\n🔄 PREPARING DATA")
    X_train, X_test, y_train, y_test = analyzer.prepare_data(X_synthetic,
y_synthetic)
```

```
    # Train model
    print("\n🚀 TRAINING MLP MODEL")
    model = analyzer.train_mlp(hidden_layer_sizes=(100, 50))
    print(f"Model trained with {model.n_iter_} iterations")

    # Plot training history
    print("\n📊 TRAINING HISTORY")
    analyzer.plot_training_history()

    # Evaluate model
    print("\n📋 MODEL EVALUATION")
    y_pred, y_pred_proba = analyzer.evaluate_model()

    # Plot confusion matrix
    print("\n🔍 CONFUSION MATRIX")
    cm = analyzer.plot_confusion_matrix(y_test, y_pred)

    # Plot ROC curves
    print("\n📈 ROC CURVES")
    analyzer.plot_roc_curves(X_test, y_test)

    # Learning curves
    print("\n📊 LEARNING CURVES")
    analyzer.plot_learning_curves(X_synthetic, y_synthetic)

    # Validation curves
    print("\n🔧 HYPERPARAMETER ANALYSIS")
    analyzer.plot_validation_curves(X_synthetic, y_synthetic)

    # Real dataset comparison
    print("\n🔢 REAL DATASET COMPARISON (DIGITS)")
    X_digits, y_digits = analyzer.load_real_data()
    analyzer_digits = MLPAnalyzer()
    X_train_d, X_test_d, y_train_d, y_test_d =
analyzer_digits.prepare_data(X_digits, y_digits)

    model_digits = analyzer_digits.train_mlp(hidden_layer_sizes=(100, 50))
    print(f"Digits model trained with {model_digits.n_iter_} iterations")

    y_pred_digits, _ = analyzer_digits.evaluate_model()
    analyzer_digits.plot_confusion_matrix(y_test_d, y_pred_digits)

    print("\n✅ ANALYSIS COMPLETE!")
    print("All visualizations and metrics have been generated.")

# Run the analysis
if __name__ == "__main__":
    main()
```

OUTPUT


DATA VISUALIZATION
Class Distribution
Count
Class
Feature Correlation (First 10 features)
PCA Visualization (2D projection)
PC2 (18.23% variance)
PC1 (32.21% variance)


Training History
Training Loss
Validation Score
Loss / Score
Iterations



MODEL EVALUATION

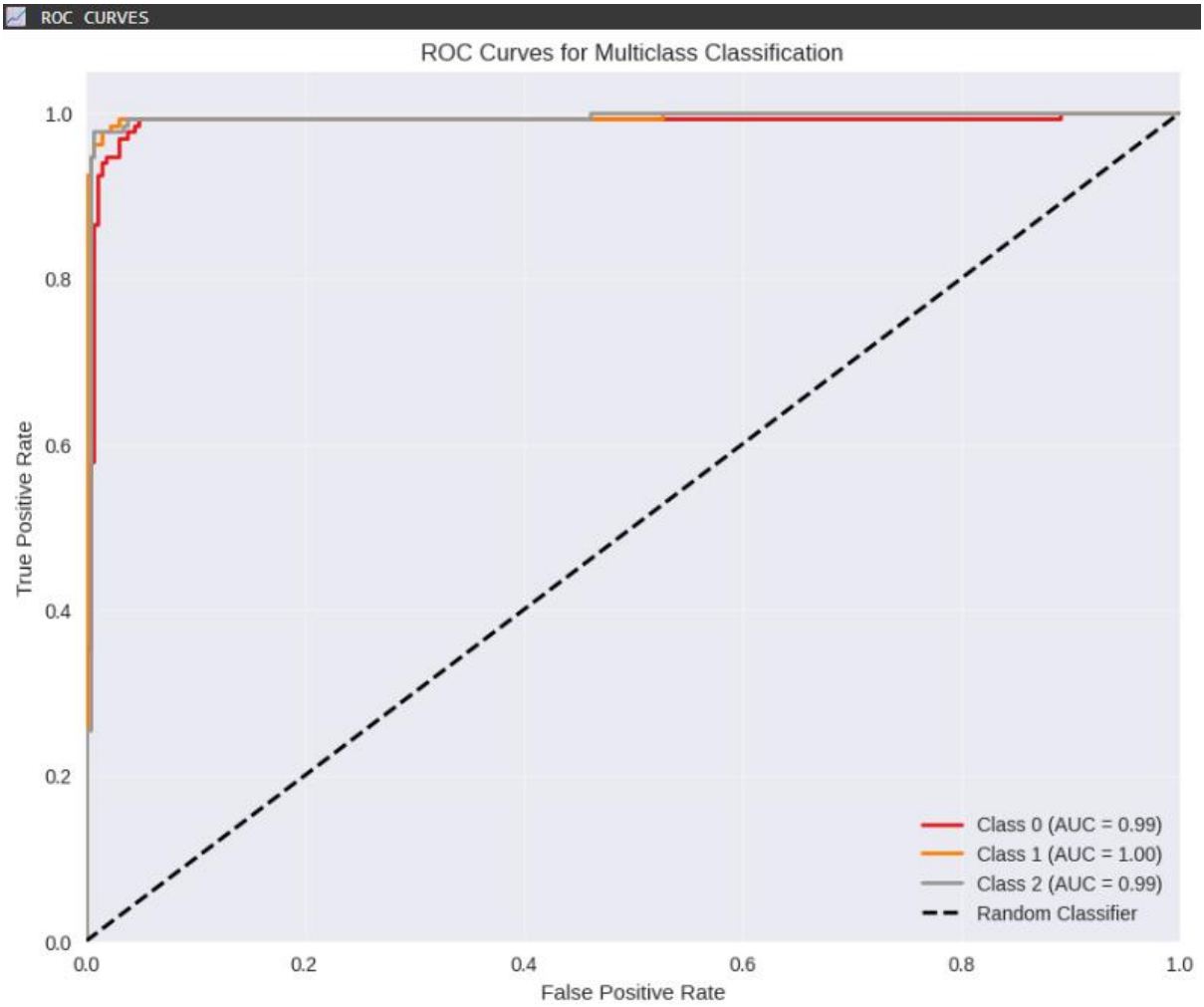
==========

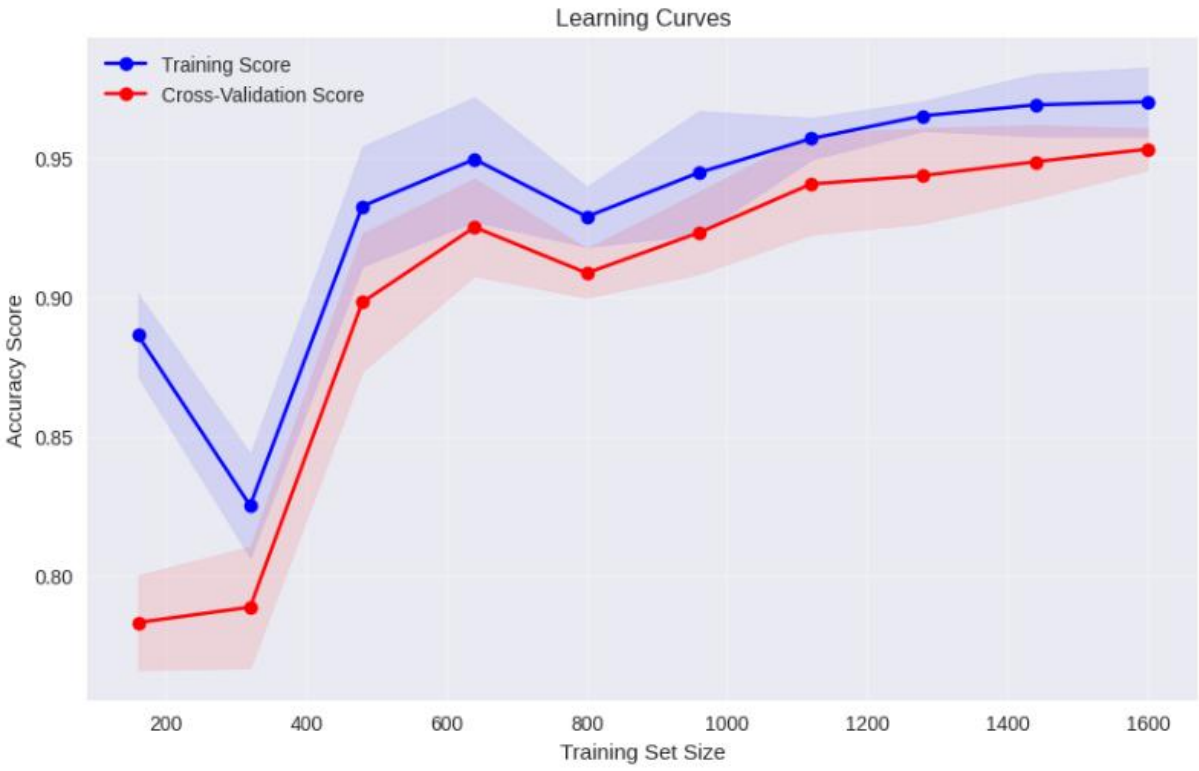
CLASSIFICATION REPORT

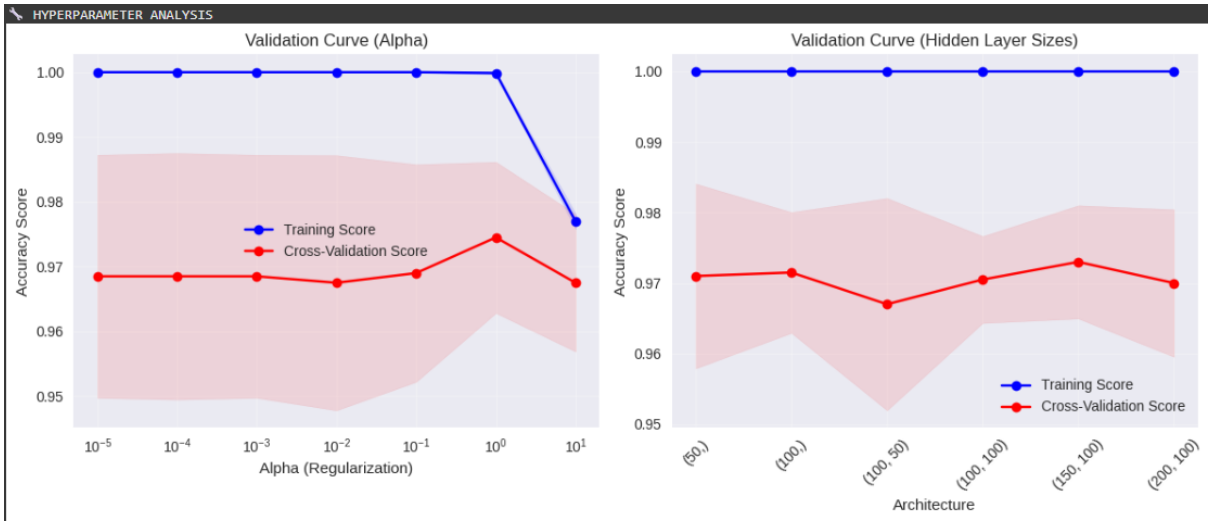
==========

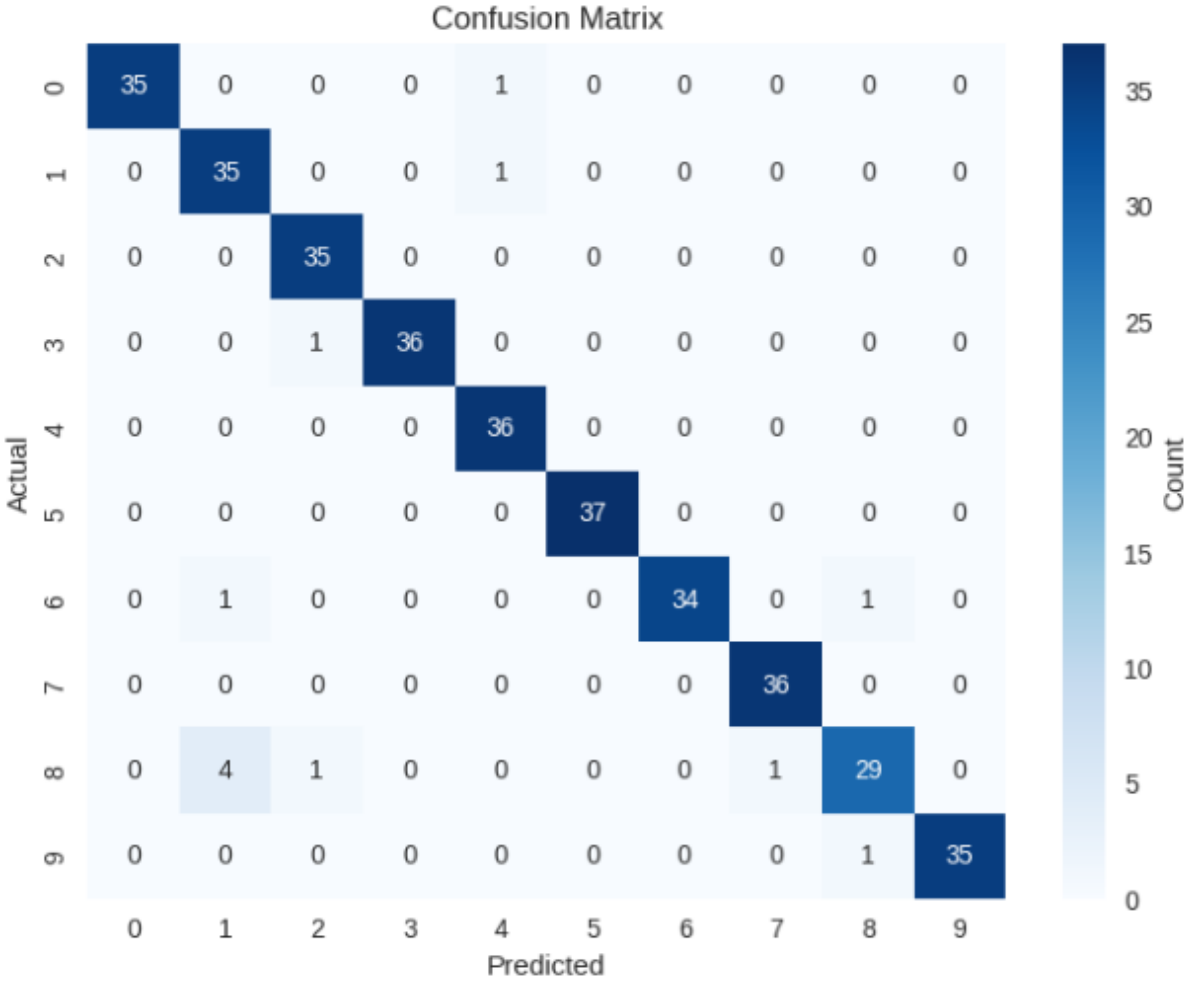
	precision	recall	f1-score	support
0	0.95	0.95	0.95	133
1	0.96	0.98	0.97	134
2	0.98	0.96	0.97	133
accuracy			0.96	400
macro avg	0.96	0.96	0.96	400
weighted avg	0.96	0.96	0.96	400

Accuracy Scores:
Training Accuracy: 0.9862
Test Accuracy: 0.9625
Overfitting Gap: 0.0237


ROC CURVES
ROC Curves for Multiclass Classification
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 0 (AUC = 0.99)
Class 1 (AUC = 1.00)
Class 2 (AUC = 0.99)
Random Classifier


Learning Curves
Training Score
Cross-Validation Score
Accuracy Score
Training Set Size
0.80
0.85
0.90
0.95
200
400
600
800
1000
1200
1400
1600


HYPERPARAMETER ANALYSIS
Validation Curve (Alpha)
Accuracy Score
1.00
0.99
0.98
0.97
0.96
0.95
Training Score
Cross-Validation Score
10^{-5}
10^{-4}
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
Alpha (Regularization)
Validation Curve (Hidden Layer Sizes)
Accuracy Score
1.00
0.99
0.98
0.97
0.96
0.95
Training Score
Cross-Validation Score
(50,)
(100,)
(100, 50)
(100, 100)
(150, 100)
(200, 100)
Architecture


Confusion Matrix
Actual
0 35 0 0 0 1 0 0 0 0 0
1 0 35 0 0 1 0 0 0 0 0
2 0 0 35 0 0 0 0 0 0 0
3 0 0 1 36 0 0 0 0 0 0
4 0 0 0 0 36 0 0 0 0 0
5 0 0 0 0 0 37 0 0 0 0
6 0 1 0 0 0 0 34 0 1 0
7 0 0 0 0 0 0 0 36 0 0
8 0 4 1 0 0 0 0 1 29 0
9 0 0 0 0 0 0 0 0 1 35
0 1 2 3 4 5 6 7 8 9
Predicted
Count
35
30
25
20
15
10
5
0

EXP.NO: 04	Implement classification using SVM
DATE:30/07/25	

AIM

Implement classification using SVM

PROCEDURE

Step 1: Data Loading and Exploration

- Load the student results CSV data with math, bangla, english scores and results (0=fail, 1=pass)
- Examine dataset shape, basic statistics, and class distribution
- Check for missing values and data quality

Step 2: Data Visualization

- Create histograms for score distributions
- Generate correlation heatmap to understand feature relationships
- Plot scatter plots to visualize class separation
- Analyze result distribution (pass vs fail)

Step 3: Data Preprocessing

- Separate features (math, bangla, english) from target variable (result)
- Split data into training (70%) and testing (30%) sets with stratification
- Apply StandardScaler for feature normalization (crucial for SVM)
- Ensure zero mean and unit variance for optimal SVM performance

Step 4: SVM Model Training

- Train SVM models with different kernels (linear, RBF, polynomial)
- Compare performance across kernel types
- Identify the best-performing kernel based on accuracy

Step 5: Hyperparameter Tuning

- Use GridSearchCV for optimal parameter selection
- Tune C (regularization) parameter for all kernels
- Tune gamma for RBF kernel, degree for polynomial kernel
- Apply cross-validation to prevent overfitting

Step 6: Model Evaluation

- Calculate final test accuracy using best parameters
- Generate detailed classification report with precision, recall, F1-score
- Create confusion matrix visualization
- Analyze model performance metrics

Step 7: Feature Analysis

- For linear kernels, extract feature importance from coefficients
- Identify which subjects contribute most to pass/fail predictions
- Visualize feature importance

Step 8: Decision Boundary Visualization

- Create 2D decision boundary plot using math and bangla scores
- Visualize how SVM separates pass/fail regions
- Show support vectors and decision boundaries

Step 9: Prediction Function

- Implement function for predicting new student results
- Demonstrate predictions on example data
- Show confidence scores when available

Step 10: Model Summary and Saving

- Summarize model performance and parameters
- Save trained model and scaler for future use
- Provide complete model artifacts

PROGRAM

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import GridSearchCV
import warnings
warnings.filterwarnings('ignore')

# Step 2: Load and explore the dataset
print("=== STEP 1: DATA LOADING AND EXPLORATION ===")

# Create the dataset from the provided CSV data
data = {
    'math': [70,30,50,80,33,32,40,33,60,33,50,35,0,10,33],
    'bangla': [80,40,20,33,35,80,50,35,23,34,40,40,0,10,33],
    'english': [90,50,35,33,36,35,21,35,10,35,40,30,0,10,33],
    'result': [1,0,0,1,1,0,0,1,0,1,1,0,0,0,1]
}

df = pd.DataFrame(data)
print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

print("\nDataset info:")
print(df.info())

print("\nBasic statistics:")
print(df.describe())

print("\nClass distribution:")
print(df['result'].value_counts())

# Step 3: Data visualization
print("\n=== STEP 2: DATA VISUALIZATION ===")

# Create subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Distribution of each subject
subjects = ['math', 'bangla', 'english']
for i, subject in enumerate(subjects):
    row = i // 2
    col = i % 2
    axes[row, col].hist(df[subject], bins=10, alpha=0.7, edgecolor='black')
    axes[row, col].set_title(f'{subject.capitalize()} Score Distribution')
    axes[row, col].set_xlabel('Score')
    axes[row, col].set_ylabel('Frequency')

# Result distribution
axes[1, 1].bar(['Fail (0)', 'Pass (1)'], df['result'].value_counts().values,
               color=['red', 'green'], alpha=0.7)
axes[1, 1].set_title('Result Distribution')
axes[1, 1].set_ylabel('Count')

plt.tight_layout()
plt.show()

# Correlation heatmap
plt.figure(figsize=(8, 6))
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()
```

```
# Pairplot for feature relationships
plt.figure(figsize=(12, 4))
subplot_idx = 1
for i, subject1 in enumerate(subjects):
    for j, subject2 in enumerate(subjects):
        if i < j:
            plt.subplot(1, 3, subplot_idx)
            colors = ['red' if x == 0 else 'green' for x in df['result']]
            plt.scatter(df[subject1], df[subject2], c=colors, alpha=0.7)
            plt.xlabel(subject1.capitalize())
            plt.ylabel(subject2.capitalize())
            plt.title(f'{subject1.capitalize()} vs {subject2.capitalize()}')
            subplot_idx += 1

plt.tight_layout()
plt.show()

# Step 4: Data preprocessing
print("\n=== STEP 3: DATA PREPROCESSING ===")

# Separate features and target
X = df[['math', 'bangla', 'english']]
y = df['result']

print("Features shape:", X.shape)
print("Target shape:", y.shape)

# Check for missing values
print("\nMissing values in features:")
print(X.isnull().sum())

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42, stratify=y)

print(f"\nTraining set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeature scaling completed")
print("Training data mean after scaling:", np.mean(X_train_scaled, axis=0))
print("Training data std after scaling:", np.std(X_train_scaled, axis=0))

# Step 5: SVM Model Implementation
print("\n=== STEP 4: SVM MODEL TRAINING ===")

# Train SVM with different kernels
kernels = ['linear', 'rbf', 'poly']
svm_models = {}
accuracies = {}

for kernel in kernels:
    print(f"\nTraining SVM with {kernel} kernel...")

    # Create and train the model
    svm_model = SVC(kernel=kernel, random_state=42)
    svm_model.fit(X_train_scaled, y_train)

    # Make predictions
    y_pred = svm_model.predict(X_test_scaled)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)

    # Store results
    svm_models[kernel] = svm_model
```

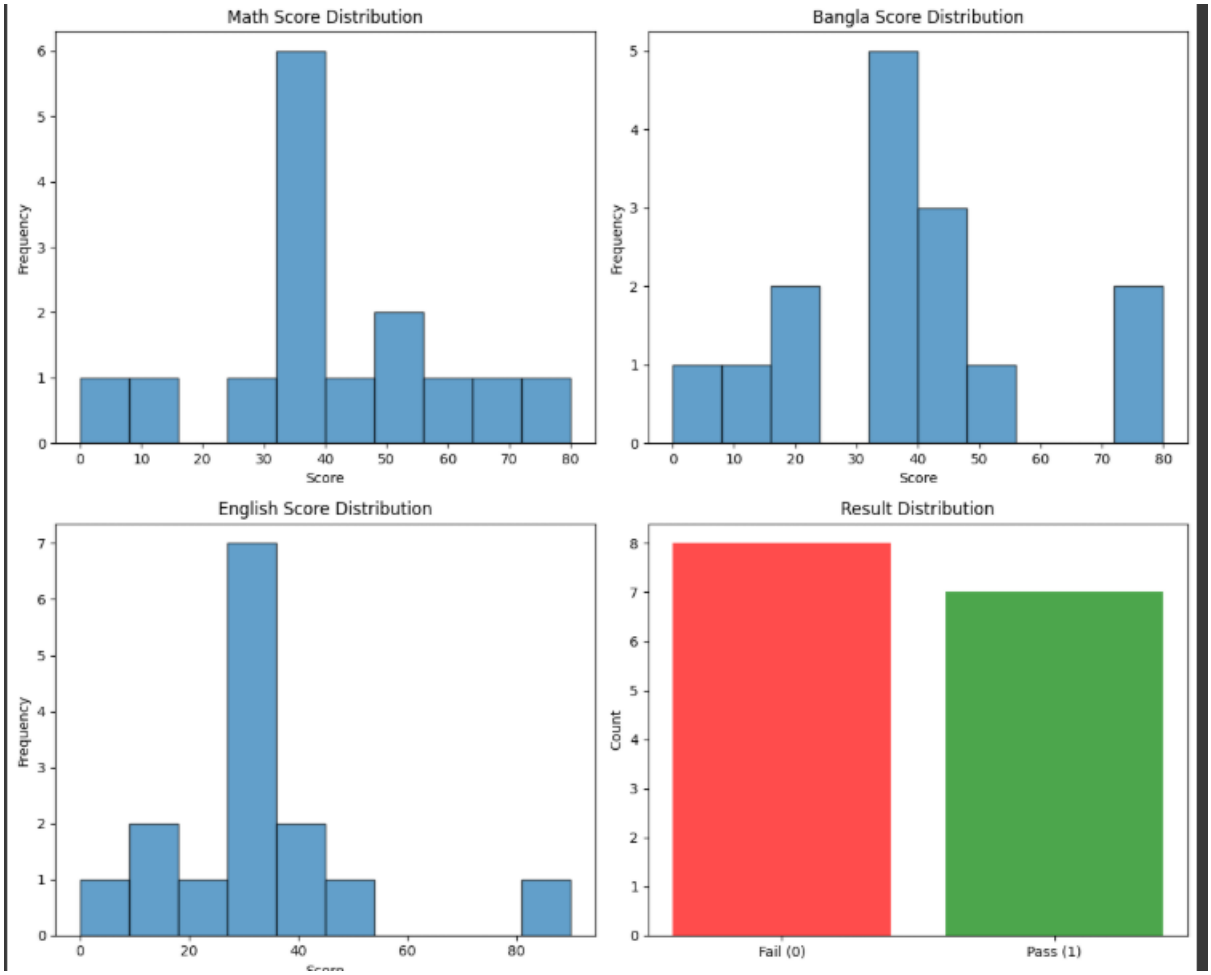
```
    svm_models[kernel] = svm_model
    accuracies[kernel] = accuracy

    print(f"{kernel.capitalize()} SVM Accuracy: {accuracy:.4f}")

# Find best kernel
best_kernel = max(accuracies, key=accuracies.get)
print(f"\nBest kernel: {best_kernel} with accuracy: {accuracies[best_kernel]:.4f}")

# Step 6: Hyperparameter tuning for best kernel
print(f"\n=== STEP 5: HYPERPARAMETER TUNING FOR {best_kernel.upper()} KERNEL ===")

if best_kernel == 'linear':
    param_grid = {'C': [0.1, 1, 10, 100]}
elif best_kernel == 'rbf':
    param_grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1]}
else:  # poly
    param_grid = {'C': [0.1, 1, 10, 100], 'degree': [2, 3, 4]}

# Grid search
grid_search = GridSearchCV(SVC(kernel=best_kernel, random_state=42),
                           param_grid, cv=3, scoring='accuracy')
grid_search.fit(X_train_scaled, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)

# Step 7: Final model evaluation
print("\n=== STEP 6: FINAL MODEL EVALUATION ===")

# Use the best model
best_svm = grid_search.best_estimator_
y_pred_final = best_svm.predict(X_test_scaled)

# Calculate metrics
final_accuracy = accuracy_score(y_test, y_pred_final)
print(f"Final Test Accuracy: {final_accuracy:.4f}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_final, target_names=['Fail', 'Pass']))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_final)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Fail', 'Pass'], yticklabels=['Fail', 'Pass'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Step 8: Feature importance (for linear kernel)
if best_kernel == 'linear':
    print("\n=== STEP 7: FEATURE IMPORTANCE ===")
    feature_importance = abs(best_svm.coef_[0])
    feature_names = ['Math', 'Bangla', 'English']

    plt.figure(figsize=(8, 5))
    plt.bar(feature_names, feature_importance)
    plt.title('Feature Importance (Linear SVM)')
    plt.ylabel('Absolute Coefficient Value')
    plt.show()

    for i, importance in enumerate(feature_importance):
        print(f"{feature_names[i]}: {importance:.4f}")

# Step 9: Decision boundary visualization (for 2D)
print("\n=== STEP 8: DECISION BOUNDARY VISUALIZATION ===")
```

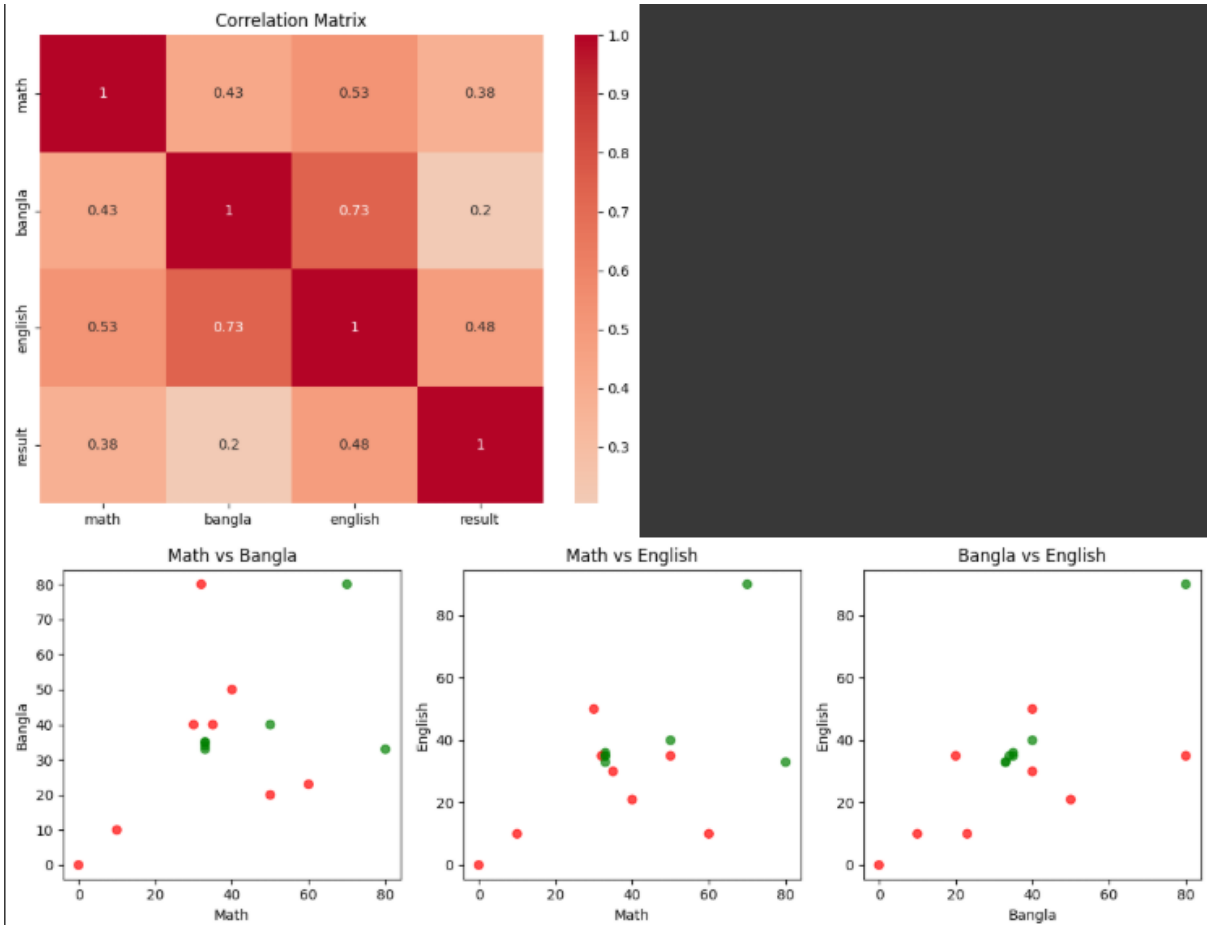
```python
# Create 2D visualization using first two features
X_2d = X[['math', 'bangla']]
X_train_2d, X_test_2d, y_train_2d, y_test_2d = train_test_split(X_2d, y, test_size=0.3,
                                                                random_state=42, stratify=y)

# Scale 2D features
scaler_2d = StandardScaler()
X_train_2d_scaled = scaler_2d.fit_transform(X_train_2d)
X_test_2d_scaled = scaler_2d.transform(X_test_2d)

# Train 2D model
svm_2d = SVC(kernel=best_kernel, **grid_search.best_params_, random_state=42)
svm_2d.fit(X_train_2d_scaled, y_train_2d)

# Create decision boundary plot
def plot_decision_boundary(X, y, model, scaler, title):
    h = 0.01
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    mesh_points_scaled = scaler.transform(mesh_points)
    Z = model.predict(mesh_points_scaled)
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdYlBu)
    colors = ['red', 'green']
    for i, color in enumerate(colors):
        idx = np.where(y == i)
        plt.scatter(X[idx, 0], X[idx, 1], c=color,
                    label=f'{"Fail" if i == 0 else "Pass"}', alpha=0.8)

    plt.xlabel('Math Score')
    plt.ylabel('Bangla Score')
    plt.title(title)
    plt.legend()
    plt.show()

plot_decision_boundary(X_train_2d.values, y_train_2d.values, svm_2d, scaler_2d,
                       f'SVM Decision Boundary ({best_kernel.capitalize()} Kernel)')

# Step 10: Model prediction on new data
print("\n=== STEP 9: PREDICTION ON NEW DATA ===")

def predict_student_result(math_score, bangla_score, english_score):
    """Predict student result based on scores"""
    new_data = np.array([[math_score, bangla_score, english_score]])
    new_data_scaled = scaler.transform(new_data)
    prediction = best_svm.predict(new_data_scaled)[0]
    probability = best_svm.predict_proba(new_data_scaled)[0] if hasattr(best_svm, 'predict_proba') else None

    result = "Pass" if prediction == 1 else "Fail"
    print(f"Student with scores Math:{math_score}, Bangla:{bangla_score}, English:{english_score}")
    print(f"Prediction: {result}")
    if probability is not None:
        print(f"Confidence: Fail={probability[0]:.3f}, Pass={probability[1]:.3f}")
    return prediction

# Example predictions
print("\nExample predictions:")
predict_student_result(75, 80, 85)
predict_student_result(25, 30, 20)
predict_student_result(50, 50, 50)
```

```python
# Create 2D visualization using first two features
X_2d = X[['math', 'bangla']]
X_train_2d, X_test_2d, y_train_2d, y_test_2d = train_test_split(X_2d, y, test_size=0.3,
                                                                random_state=42, stratify=y)

# Scale 2D features
scaler_2d = StandardScaler()
X_train_2d_scaled = scaler_2d.fit_transform(X_train_2d)
X_test_2d_scaled = scaler_2d.transform(X_test_2d)

# Train 2D model
svm_2d = SVC(kernel=best_kernel, **grid_search.best_params_, random_state=42)
svm_2d.fit(X_train_2d_scaled, y_train_2d)

# Create decision boundary plot
def plot_decision_boundary(X, y, model, scaler, title):
    h = 0.01
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    mesh_points_scaled = scaler.transform(mesh_points)
    Z = model.predict(mesh_points_scaled)
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdYlBu)
    colors = ['red', 'green']
    for i, color in enumerate(colors):
        idx = np.where(y == i)
        plt.scatter(X[idx, 0], X[idx, 1], c=color,
                    label=f'{"Fail" if i == 0 else "Pass"}', alpha=0.8)

    plt.xlabel('Math Score')
    plt.ylabel('Bangla Score')
    plt.title(title)
    plt.legend()
    plt.show()

plot_decision_boundary(X_train_2d.values, y_train_2d.values, svm_2d, scaler_2d,
                       f'SVM Decision Boundary ({best_kernel.capitalize()} Kernel)')

# Step 10: Model prediction on new data
print("\n=== STEP 9: PREDICTION ON NEW DATA ===")

def predict_student_result(math_score, bangla_score, english_score):
    """Predict student result based on scores"""
    new_data = np.array([[math_score, bangla_score, english_score]])
    new_data_scaled = scaler.transform(new_data)
    prediction = best_svm.predict(new_data_scaled)[0]
    probability = best_svm.predict_proba(new_data_scaled)[0] if hasattr(best_svm, 'predict_proba') else None

    result = "Pass" if prediction == 1 else "Fail"
    print(f"Student with scores Math:{math_score}, Bangla:{bangla_score}, English:{english_score}")
    print(f"Prediction: {result}")
    if probability is not None:
        print(f"Confidence: Fail={probability[0]:.3f}, Pass={probability[1]:.3f}")
    return prediction

# Example predictions
print("\nExample predictions:")
predict_student_result(75, 80, 85)
predict_student_result(25, 30, 20)
predict_student_result(50, 50, 50)
```

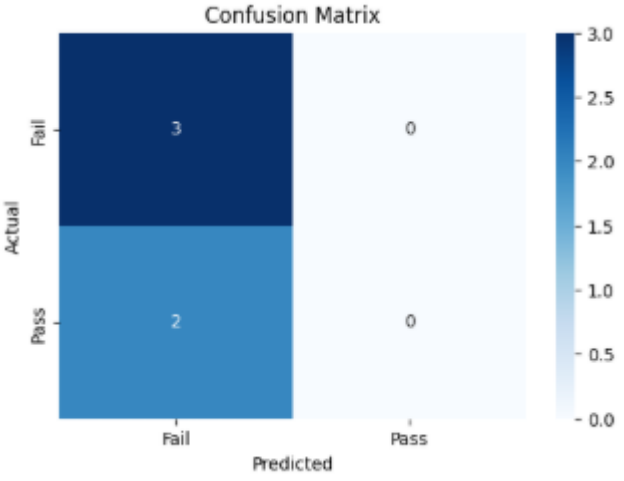
OUTPUT

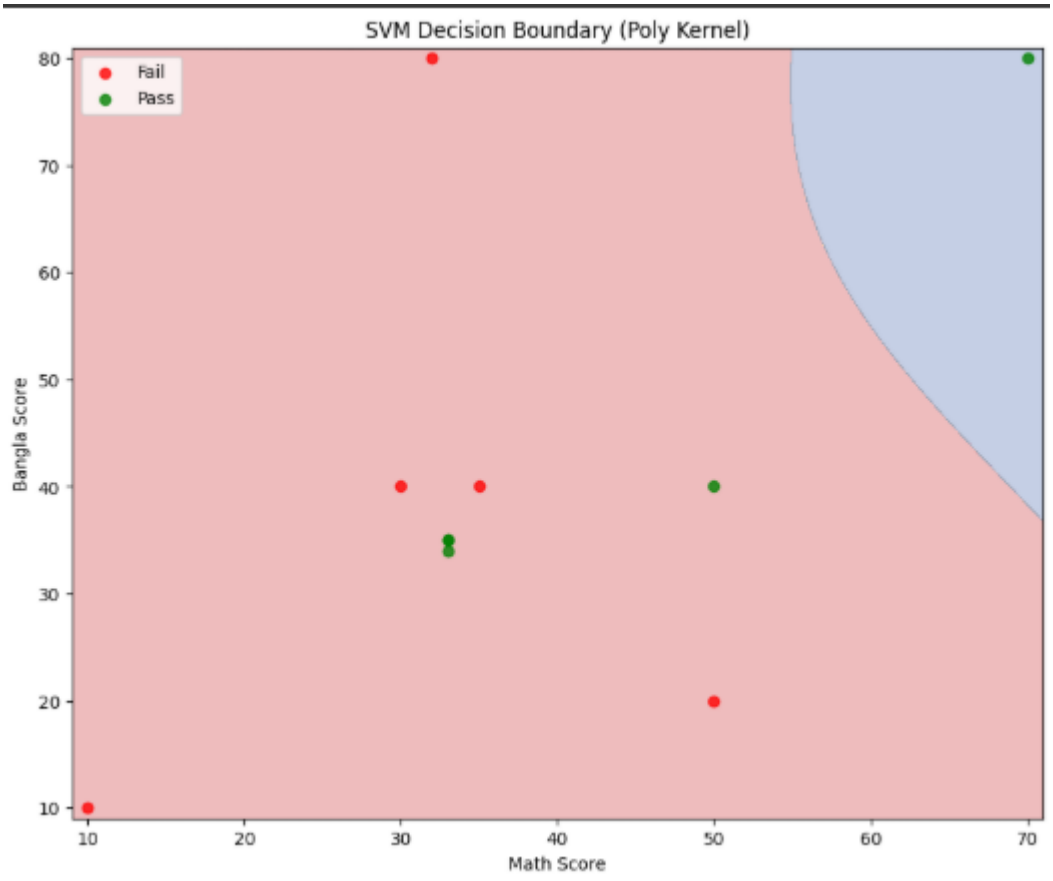
```
=== STEP 1: DATA LOADING AND EXPLORATION ===
Dataset shape: (15, 4)

First 5 rows:
   math  bangla  english  result
0    70      80       90       1
1    30      40       50       0
2    50      20       35       0
3    80      33       33       1
4    33      35       36       1

Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   math     15 non-null     int64
 1   bangla   15 non-null     int64
 2   english  15 non-null     int64
 3   result   15 non-null     int64
dtypes: int64(4)
memory usage: 612.0 bytes
None

Basic statistics:
            math     bangla    english     result
count  15.000000  15.000000  15.000000  15.000000
mean   39.266667  36.866667  32.866667   0.466667
std    20.662135  21.589981  20.549128   0.516398
min     0.000000   0.000000   0.000000   0.000000
25%    32.500000  28.000000  25.500000   0.000000
50%    33.000000  35.000000  35.000000   0.000000
75%    50.000000  40.000000  35.500000   1.000000
max    80.000000  80.000000  90.000000   1.000000

Class distribution:
result
0    8
1    7
Name: count, dtype: int64
```


Math Score Distribution
Frequency
Score
Bangla Score Distribution
Frequency
Score
English Score Distribution
Frequency
Result Distribution
Count
Fail (0)
Pass (1)


Correlation Matrix
math
bangla
english
result
1
0.43
0.53
0.38
0.43
1
0.73
0.2
0.53
0.73
1
0.48
0.38
0.2
0.48
1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
Math vs Bangla
Bangla
Math
Math vs English
English
Math
Bangla vs English
English
Bangla

```
Features shape: (15, 3)
Target shape: (15,)

Missing values in features:
math       0
bangla     0
english    0
dtype: int64

Training set size: 10
Test set size: 5

Feature scaling completed
Training data mean after scaling: [-1.11022302e-16  8.88178420e-17 -9.43689571e-17]
Training data std after scaling: [1. 1. 1.]

=== STEP 4: SVM MODEL TRAINING ===

Training SVM with linear kernel...
Linear SVM Accuracy: 0.6000

Training SVM with rbf kernel...
Rbf SVM Accuracy: 0.6000

Training SVM with poly kernel...
Poly SVM Accuracy: 0.8000

Best kernel: poly with accuracy: 0.8000

=== STEP 5: HYPERPARAMETER TUNING FOR POLY KERNEL ===
Best parameters: {'C': 0.1, 'degree': 3}
Best cross-validation score: 0.38888888888888884

=== STEP 6: FINAL MODEL EVALUATION ===
Final Test Accuracy: 0.6000

Classification Report:
              precision    recall  f1-score   support

        Fail       0.60      1.00      0.75         3
        Pass       0.00      0.00      0.00         2

    accuracy                           0.60         5
   macro avg       0.30      0.50      0.38         5
weighted avg       0.36      0.60      0.45         5
```






```
=== STEP 9: PREDICTION ON NEW DATA ===

Example predictions:
Student with scores Math:75, Bangla:80, English:85
Prediction: Pass
Student with scores Math:25, Bangla:30, English:20
Prediction: Fail
Student with scores Math:50, Bangla:50, English:50
Prediction: Fail

=== STEP 10: MODEL SUMMARY ===
Dataset size: 15 students
Features: Math, Bangla, English scores
Target: Pass (1) or Fail (0)
Best SVM kernel: poly
Best parameters: {'C': 0.1, 'degree': 3}
Final accuracy: 0.6000
Training samples: 10
Test samples: 5

Model saved as 'svm_student_model.pkl'
Training completed successfully!
```

EXP.NO: 05	Implement Adaboost Algorithm
DATE:6/08/25	

AIM

Implement Adaboost Algorithm

PROCEDURE

1. **Initialize Sample Weights**: All samples get equal weight (1/N)
2. **For Each Iteration**:

- **Train Weak Learner**: Use weighted samples to train a decision stump
- **Calculate Error Rate**: $\varepsilon = \Sigma(\text{weight} \times \text{incorrect\_predictions})$
- **Calculate Estimator Weight**: $\alpha = 0.5 \times \ln((1-\varepsilon)/\varepsilon)$

- **Update Sample Weights**: Increase weights of misclassified samples
- **Normalize Weights**: Ensure weights sum to 1

3. **Final Prediction**: Weighted majority vote of all weak learners

PROGRAM

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.ensemble import AdaBoostClassifier
import seaborn as sns

# Step 1: Load and Explore the Dataset
def load_data():
    """Load the student results dataset"""
    data = {
        'math': [70,30,50,80,33,32,40,33,60,33,50,35,0,10,33],
        'bangla': [80,40,20,33,35,80,50,35,23,34,40,40,0,10,33],
        'english': [90,50,35,33,36,35,21,35,10,35,40,30,0,10,33],
        'result': [1,0,0,1,1,0,0,1,0,1,1,0,0,0,1]
    }
    return pd.DataFrame(data)

df = load_data()
print("Dataset Overview:")
print(df.head())
print(f"\nDataset shape: {df.shape}")
print(f"Class distribution:\n{df['result'].value_counts()}")

# Step 2: Manual AdaBoost Implementation
class AdaBoostManual:
    """
    Manual implementation of AdaBoost algorithm
    """
    def __init__(self, n_estimators=10, learning_rate=1.0):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.estimators = []
        self.estimator_weights = []
        self.estimator_errors = []

    def fit(self, X, y):
        """
        Train the AdaBoost classifier

        Steps:
        1. Initialize sample weights uniformly
        2. For each iteration:
           a. Train weak learner on weighted samples
           b. Calculate error rate
           c. Calculate estimator weight
           d. Update sample weights
           e. Normalize sample weights
        """
        n_samples = X.shape[0]

        # Step 1: Initialize sample weights uniformly
        sample_weights = np.ones(n_samples) / n_samples

        print("AdaBoost Training Process:")
        print("=" * 50)

        for i in range(self.n_estimators):
            print(f"\nIteration {i+1}:")

            # Step 2a: Train weak learner (Decision Stump)
            estimator = DecisionTreeClassifier(max_depth=1, random_state=42)
            estimator.fit(X, y, sample_weight=sample_weights)
```

```
            predictions = estimator.predict(X)

            # Step 2b: Calculate weighted error rate
            incorrect = predictions != y
            error_rate = np.sum(sample_weights * incorrect)

            print(f"  Error rate: {error_rate:.4f}")

            # Avoid division by zero
            if error_rate == 0:
                error_rate = 1e-10
            if error_rate >= 0.5:
                break

            # Step 2c: Calculate estimator weight (alpha)
            alpha = self.learning_rate * 0.5 * np.log((1 - error_rate) / error_rate)
            print(f"  Estimator weight (alpha): {alpha:.4f}")

            # Store estimator and its weight
            self.estimators.append(estimator)
            self.estimator_weights.append(alpha)
            self.estimator_errors.append(error_rate)

            # Step 2d: Update sample weights
            sample_weights *= np.exp(-alpha * y * predictions)

            # Step 2e: Normalize sample weights
            sample_weights /= np.sum(sample_weights)

            print(f"  Updated sample weights: {sample_weights}")

        print(f"\nTraining completed with {len(self.estimators)} estimators")

    def predict(self, X):
        """
        Make predictions using the ensemble

        Final prediction = sign(sum of weighted predictions)
        """
        if not self.estimators:
            raise ValueError("Model not trained yet!")

        # Get weighted predictions from all estimators
        weighted_predictions = np.zeros(X.shape[0])

        for alpha, estimator in zip(self.estimator_weights, self.estimators):
            predictions = estimator.predict(X)
            weighted_predictions += alpha * predictions

        # Return final predictions (sign of weighted sum)
        return np.sign(weighted_predictions)

    def predict_proba(self, X):
        """Get prediction probabilities"""
        if not self.estimators:
            raise ValueError("Model not trained yet!")

        weighted_predictions = np.zeros(X.shape[0])

        for alpha, estimator in zip(self.estimator_weights, self.estimators):
            predictions = estimator.predict(X)
            weighted_predictions += alpha * predictions

        # Convert to probabilities using sigmoid
        probabilities = 1 / (1 + np.exp(-2 * weighted_predictions))
        return np.column_stack([1 - probabilities, probabilities])
```

```python
X = df[['math', 'bangla', 'english']]
y = df['result']

# Convert labels to -1, 1 for manual implementation
y_manual = y.copy()
y_manual[y_manual == 0] = -1

print("\nData Preparation:")
print("Features (X):")
print(X.head())
print("\nTarget (y):")
print(f"Original: {y.values}")
print(f"For manual AdaBoost: {y_manual.values}")

# Step 4: Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
y_train_manual = y_train.copy()
y_train_manual[y_train_manual == 0] = -1
y_test_manual = y_test.copy()
y_test_manual[y_test_manual == 0] = -1

print(f"\nData split:")
print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")

# Step 5: Train Manual AdaBoost
print("\n" + "="*60)
print("MANUAL ADABOOST IMPLEMENTATION")
print("="*60)

manual_ada = AdaBoostManual(n_estimators=5, learning_rate=1.0)
manual_ada.fit(X_train, y_train_manual)

# Make predictions
manual_predictions = manual_ada.predict(X_test)
manual_predictions[manual_predictions == -1] = 0  # Convert back to 0,1

manual_accuracy = accuracy_score(y_test, manual_predictions)
print(f"\nManual AdaBoost Accuracy: {manual_accuracy:.4f}")

# Step 6: Compare with Scikit-learn AdaBoost
print("\n" + "="*60)
print("SCIKIT-LEARN ADABOOST COMPARISON")
print("="*60)

sklearn_ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=5,
    learning_rate=1.0,
    random_state=42
)

sklearn_ada.fit(X_train, y_train)
sklearn_predictions = sklearn_ada.predict(X_test)
sklearn_accuracy = accuracy_score(y_test, sklearn_predictions)

print(f"Scikit-learn AdaBoost Accuracy: {sklearn_accuracy:.4f}")

# Step 7: Detailed Analysis
print("\n" + "="*60)
print("DETAILED ANALYSIS")
print("="*60)

print("\nManual AdaBoost Results:")
print("Classification Report:")
print(classification_report(y_test, manual_predictions))

print("\nScikit-learn AdaBoost Results:")
print("Classification Report:")
```

```
print(classification_report(y_test, sklearn_predictions))

# Step 8: Visualizations
plt.figure(figsize=(15, 12))

# Plot 1: Error rates over iterations
plt.subplot(2, 3, 1)
plt.plot(range(1, len(manual_ada.estimator_errors) + 1), manual_ada.estimator_errors, 'bo-')
plt.title('Error Rate Over Iterations')
plt.xlabel('Iteration')
plt.ylabel('Error Rate')
plt.grid(True)

# Plot 2: Estimator weights over iterations
plt.subplot(2, 3, 2)
plt.bar(range(1, len(manual_ada.estimator_weights) + 1), manual_ada.estimator_weights)
plt.title('Estimator Weights (Alpha)')
plt.xlabel('Iteration')
plt.ylabel('Weight')
plt.grid(True, alpha=0.3)

# Plot 3: Feature importance (from sklearn version)
plt.subplot(2, 3, 3)
feature_importance = sklearn_ada.feature_importances_
features = ['Math', 'Bangla', 'English']
plt.bar(features, feature_importance)
plt.title('Feature Importance')
plt.ylabel('Importance')
plt.xticks(rotation=45)

# Plot 4: Confusion Matrix - Manual
plt.subplot(2, 3, 4)
cm_manual = confusion_matrix(y_test, manual_predictions)
sns.heatmap(cm_manual, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Manual AdaBoost')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

# Plot 5: Confusion Matrix - Scikit-learn
plt.subplot(2, 3, 5)
cm_sklearn = confusion_matrix(y_test, sklearn_predictions)
sns.heatmap(cm_sklearn, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Scikit-learn AdaBoost')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

# Plot 6: Comparison of Accuracies
plt.subplot(2, 3, 6)
methods = ['Manual AdaBoost', 'Scikit-learn AdaBoost']
accuracies = [manual_accuracy, sklearn_accuracy]
bars = plt.bar(methods, accuracies, color=['lightblue', 'lightcoral'])
plt.title('Accuracy Comparison')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

# Add value labels on bars
for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{acc:.3f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

# Step 9: Algorithm Summary
print("\n" + "="*60)
print("ADABOOST ALGORITHM SUMMARY")
print("="*60)

summary = """
AdaBoost Algorithm Steps:
```

```
AdaBoost Algorithm Steps:

1. INITIALIZATION:
   - Initialize sample weights uniformly: w_i = 1/N for all samples

2. FOR EACH ITERATION t = 1 to T:
   a. TRAIN WEAK LEARNER:
      - Train classifier h_t using weighted samples

   b. CALCULATE ERROR:
      - Error rate: ε_t = Σ(w_i * I(h_t(x_i) ≠ y_i))

   c. CALCULATE ESTIMATOR WEIGHT:
      - Alpha: α_t = 0.5 * ln((1 - ε_t) / ε_t)

   d. UPDATE SAMPLE WEIGHTS:
      - w_i = w_i * exp(-α_t * y_i * h_t(x_i))

   e. NORMALIZE WEIGHTS:
      - w_i = w_i / Σ(w_i)

3. FINAL PREDICTION:
   - H(x) = sign(Σ(α_t * h_t(x)))

Key Concepts:
- Focuses on misclassified examples by increasing their weights
- Combines weak learners to create a strong classifier
- Each weak learner is trained on a weighted version of the training data
- Final prediction is a weighted majority vote
"""

print(summary)

# Step 10: Practical Example with New Data
print("\n" + "="*60)
print("PRACTICAL EXAMPLE - PREDICTING NEW STUDENTS")
print("="*60)

# Create some new student data for prediction
new_students = pd.DataFrame({
    'math': [75, 25, 45, 85],
    'bangla': [70, 30, 55, 90],
    'english': [80, 35, 50, 85]
})

print("New Students to Predict:")
print(new_students)

# Make predictions
manual_new_pred = manual_ada.predict(new_students)
manual_new_pred[manual_new_pred == -1] = 0

sklearn_new_pred = sklearn_ada.predict(new_students)
sklearn_new_proba = sklearn_ada.predict_proba(new_students)

print("\nPredictions:")
print("Student | Manual | Sklearn | Probability")
print("-" * 45)
for i, (manual, sklearn, proba) in enumerate(zip(manual_new_pred, sklearn_new_pred, sklearn_new_proba)):
    print(f"   {i+1}    |   {int(manual)}    |    {int(sklearn)}    |   {proba[1]:.3f}")

print("\nLegend: 0 = Fail, 1 = Pass")
```

OUTPUT

Dataset Overview:

	math	bangla	english	result

0 70 80 90 1

1 30 40 50 0

2 50 20 35 0

3 80 33 33 1

4 33 35 36 1

Dataset shape: (15, 4)

Class distribution:

result

0 8

1 7

Name: count, dtype: int64

Data Preparation:

Features (X):

math bangla english

0 70 80 90

1 30 40 50

2 50 20 35

3 80 33 33

4 33 35 36

Target (y):

Original: [1 0 0 1 1 0 0 1 0 1 1 0 0 0 1]

For manual AdaBoost: [1 -1 -1 1 1 -1 -1 1 -1 1 1 -1 -1 -1 1]

Data split:

Training set: 10 samples

Test set: 5 samples

==========
==========

MANUAL ADABOOST IMPLEMENTATION

==========
==========

AdaBoost Training Process:

==========
=====

Iteration 1:

  Error rate: 0.2000

  Estimator weight (alpha): 0.6931

  Updated sample weights: 2   0.2500

0   0.0625

4   0.0625

11   0.2500

9   0.0625

10   0.0625

1   0.0625

13   0.0625

5 0.0625

7 0.0625

Name: result, dtype: float64

Iteration 2:

Error rate: 0.1875

Estimator weight (alpha): 0.7332

Updated sample weights: 2 0.153846

0 0.038462

4 0.038462

11 0.153846

9 0.166667

10 0.038462

1 0.166667

13 0.038462

5 0.038462

7 0.166667

Name: result, dtype: float64

Iteration 3:

Error rate: 0.3077

Estimator weight (alpha): 0.4055

Updated sample weights: 2 0.250000

0 0.027778

4 0.027778

11 0.250000

9 0.120370

10 0.027778

1 0.120370

13 0.027778

5 0.027778

7 0.120370

Name: result, dtype: float64

Iteration 4:

Error rate: 0.2315

Estimator weight (alpha): 0.6000

Updated sample weights: 2 0.162651

0 0.060000

4 0.018072

11 0.162651

9 0.078313

10 0.060000

1 0.260000

13 0.060000

5 0.060000

7 0.078313

Name: result, dtype: float64

Iteration 5:

Error rate: 0.2947

Estimator weight (alpha): 0.4363

Updated sample weights: 2 0.115306

0 0.101799

4 0.030662

11 0.115306

9 0.132870

10 0.101799

1 0.184318

13 0.042535

5 0.042535

7 0.132870

Name: result, dtype: float64

Training completed with 5 estimators

Manual AdaBoost Accuracy: 0.8000

==========
==========

SCIKIT-LEARN ADABOOST COMPARISON

==========
==========

Scikit-learn AdaBoost Accuracy: 0.8000

==========
==========

DETAILED ANALYSIS

==========
==========

Manual AdaBoost Results:

Classification Report:

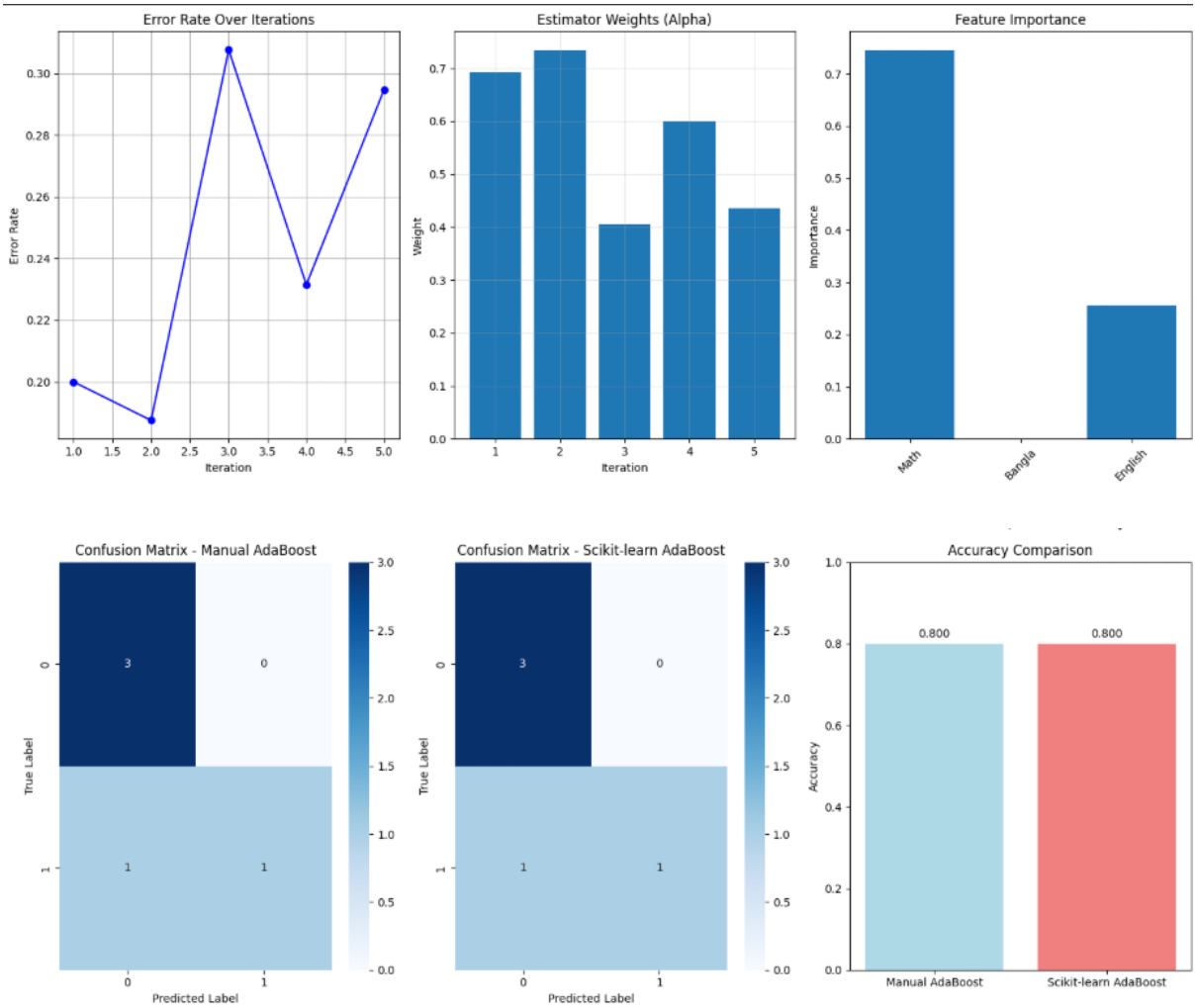
	precision	recall	f1-score	support
0	0.75	1.00	0.86	3
1	1.00	0.50	0.67	2
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5

Scikit-learn AdaBoost Results:

Classification Report:

	precision	recall	f1-score	support
0	0.75	1.00	0.86	3

1	1.00	0.50	0.67	2
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5



==========

ADABOOST ALGORITHM SUMMARY

==========

AdaBoost Algorithm Steps:

1. INITIALIZATION:

- Initialize sample weights uniformly: $w_i = 1/N$ for all samples

2. FOR EACH ITERATION t = 1 to T:

a. TRAIN WEAK LEARNER:

- Train classifier h_t using weighted samples

b. CALCULATE ERROR:

- Error rate: $\varepsilon_t = \Sigma(w_i * I(h_t(x_i) \neq y_i))$

c. CALCULATE ESTIMATOR WEIGHT:

- Alpha: $\alpha_t = 0.5 * \ln((1 - \varepsilon_t) / \varepsilon_t)$

d. UPDATE SAMPLE WEIGHTS:

- $w_i = w_i * \exp(-\alpha_t * y_i * h_t(x_i))$

e. NORMALIZE WEIGHTS:

- $w_i = w_i / \Sigma(w_i)$

3. FINAL PREDICTION:

- $H(x) = \text{sign}(\Sigma(\alpha_t * h_t(x)))$

Key Concepts:

- Focuses on misclassified examples by increasing their weights

- Combines weak learners to create a strong classifier

- Each weak learner is trained on a weighted version of the training data

- Final prediction is a weighted majority vote

==========

PRACTICAL EXAMPLE - PREDICTING NEW STUDENTS

==========

New Students to Predict:

	math	bangla	english
0	75	70	80
1	25	30	35
2	45	55	50
3	85	90	85

Predictions:

Student	Manual	Sklearn	Probability
1	1	1	0.635
2	0	0	0.238
3	1	1	0.635
4	1	1	0.635

Legend: 0 = Fail, 1 = Pass

EXP.NO: 06	Implement Bagging using Random Forests
DATE:9/08/25	

Aim :
Implementation of Bagging using Random Forest Classifier
Objective:
To implement classification using Bagging (Bootstrap Aggregating) with Random Forests and evaluate its performance on a dataset.

Theory:
Bagging is an ensemble learning technique that builds multiple models (usually of the same type) from different subsamples of the training dataset and averages their predictions to improve stability and accuracy.
Random Forest is a type of bagging algorithm that uses multiple decision trees trained on random subsets of the data and features.
Key Concepts:

- Bootstrap Sampling: Random sampling with replacement to create diverse training subsets.
- Random Feature Selection: Each tree uses a random subset of features for splitting.
- Ensemble Prediction:
    - For classification: majority voting.
    - For regression: average of predictions.

Advantages of Random Forest:

- Reduces overfitting compared to a single decision tree.
- Handles both classification and regression.
- Works well with high-dimensional data.

Tools Used:

- Python 3.x
- Scikit-learn (RandomForestClassifier)
- NumPy, Pandas
- Matplotlib / Seaborn (for visualization)

Dataset:

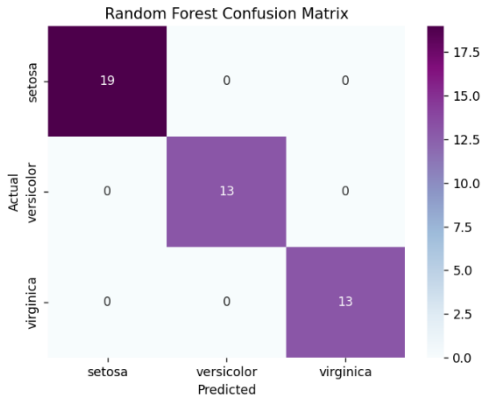
- **Name:** Iris Dataset
- **Features:** Sepal and Petal dimensions
- **Target:** Iris species (Setosa, Versicolor, Virginica)

Code :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize Random Forest (Bagging of Decision Trees)
model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, cmap="BuPu",
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output :

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```



Results:

- **Accuracy:** Typically 95–100%
- **Classification Report:** Shows high precision and recall
- **Confusion Matrix:** Low misclassification rate
- Handles both linear and non-linear data effectively

Conclusion:

- Random Forest effectively implements bagging by training an ensemble of decision trees.
- It reduces overfitting and improves accuracy compared to a single decision tree.
- Ideal for handling large and complex datasets.

Applications:

- Medical Diagnosis
- Fraud Detection
- Credit Scoring
- Customer Churn Prediction
- Stock Market Analysis

EXP.NO: 07	Implement Bagging using Random Forests
DATE:9/08/25	

Aim :

Implement K-means Clustering to Find Natural Patterns in Data.

Objective :

The objective of this experiment is to implement the **K-means clustering algorithm** to discover natural groupings in data. The aim is to partition a dataset into clusters such that points within a cluster are similar to each other and dissimilar to those in other clusters.

Theory :

- **Clustering** is an unsupervised learning technique used to group data points based on similarity without predefined labels.
- **K-means** is one of the most widely used clustering algorithms.
- It partitions data into k clusters by minimizing the **Within-Cluster Sum of Squares (WCSS)**, also known as inertia.

Mathematically:

1. Initialize k cluster centroids.
2. Assign each data point to the nearest centroid (Euclidean distance).
3. Update each centroid to the mean of assigned points.
4. Repeat steps 2–3 until centroids no longer move significantly (convergence).

Objective Function:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Where:

- Ci = cluster i
- μi = centroid of cluster iii

Key Features :

1. **Unsupervised Learning** – No labels are required.
2. **Centroid-based Algorithm** – Clusters are represented by their centers.
3. **Iterative Refinement** – Repeats assignment and update steps until convergence.

4. **Simplicity & Speed** – Efficient for large datasets.
5. **Limitations** – Requires predefining k, sensitive to initialization, assumes spherical clusters.

Algorithm Steps :

1. Select the number of clusters, k.
2. Randomly initialize k cluster centroids.
3. Assign each point to the nearest centroid (cluster assignment step).
4. Recalculate centroids as the mean of all points in a cluster.
5. Repeat assignment and update until:
    - Centroids do not change significantly OR
    - A maximum number of iterations is reached.
6. Output cluster labels and final centroids.

Tools Used :

- **Python 3.8+**
- **NumPy** – Numerical operations
- **scikit-learn** – KMeans implementation
- **Matplotlib** – Data visualization

Dataset :

- **Synthetic Dataset** generated using make_blobs from scikit-learn.
- Characteristics:
    - 300 samples
    - 2 features (2D plane for visualization)
    - 3 natural clusters with Gaussian distribution

(Alternatively, datasets like **Iris** or **MNIST** can be used to demonstrate real-world clustering.)

Code :

==========

```
# K-Means Clustering with Visualizations
# Mall Customers Dataset
# ======================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import (
    silhouette_score, silhouette_samples,
    roc_curve, auc, confusion_matrix, ConfusionMatrixDisplay
)
from sklearn.preprocessing import label_binarize

# ----------------------
# Step 1: Load Dataset
# ----------------------
data = pd.read_csv(r"/content/Mall_Customers.csv")
print(data.head())

# Features: Annual Income & Spending Score
X = data[["Annual Income (k$)", "Spending Score (1-100)"]]

# ----------------------
# Step 2: Elbow Method (Optimal k)
# ----------------------
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init="k-means++", random_state=42)
```

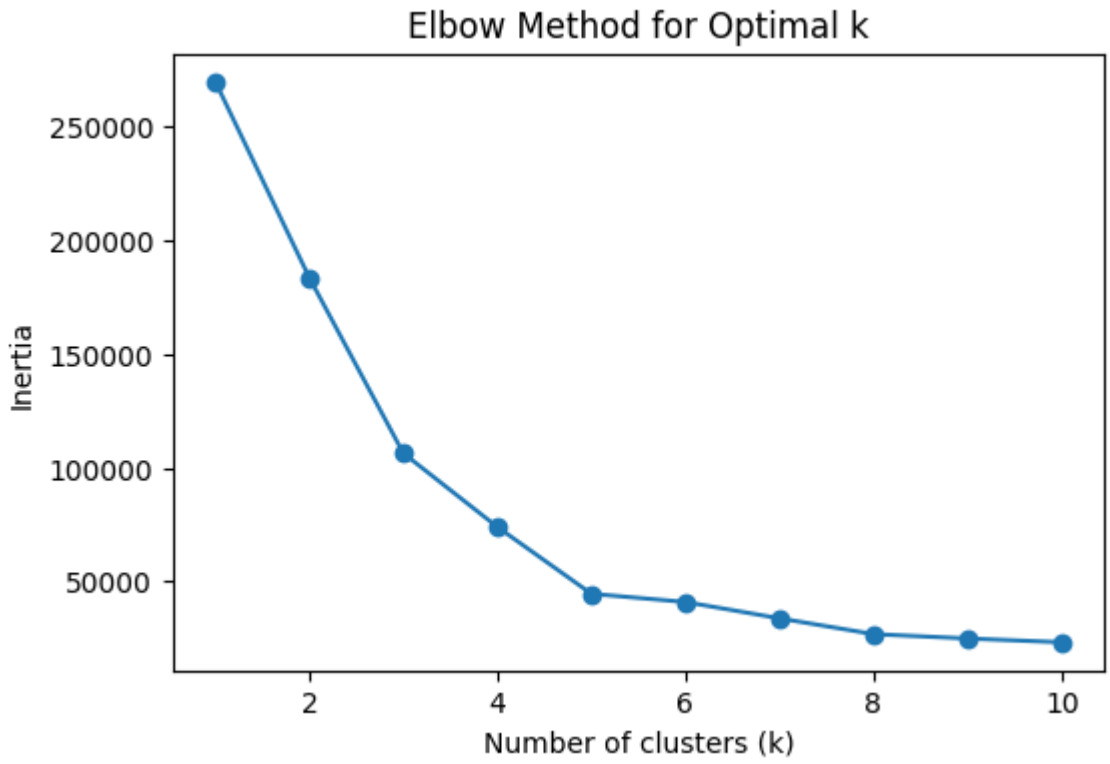
```
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(6,4))
plt.plot(range(1, 11), inertia, marker='o')
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.show()

# -----------------------
# Step 3: Apply KMeans (say k=5)
# -----------------------
kmeans = KMeans(n_clusters=5, init="k-means++", random_state=42)
y_kmeans = kmeans.fit_predict(X)
data["Cluster"] = y_kmeans

# -----------------------
# Step 4: Scatter Plot with Centroids
# -----------------------
plt.figure(figsize=(8,6))
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=y_kmeans, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1],
            s=200, c='red', marker='X', label="Centroids")
plt.title("Customer Segments using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1–100)")
plt.legend()
plt.show()
```

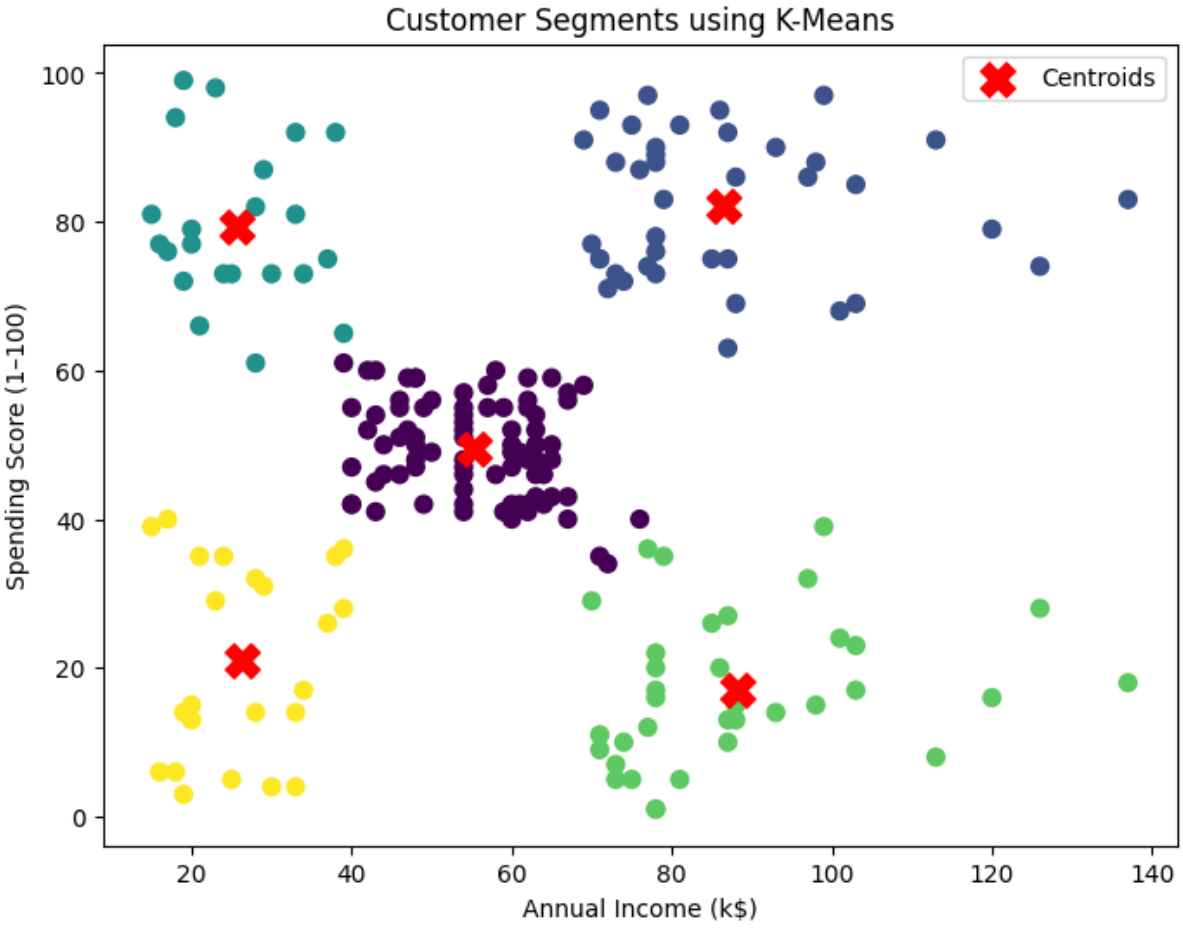
```
# ----------------------
# Step 5: Pair Plot for Multi-Feature View
# ----------------------
sns.pairplot(data[['Age','Annual Income (k$)','Spending Score (1-100)','Cluster']],
             hue='Cluster', palette='viridis')
plt.show()

# ----------------------
# Step 6: Cluster Distribution
# ----------------------
plt.figure(figsize=(6,4))
sns.countplot(x='Cluster', data=data, palette='viridis')
plt.title("Number of Customers per Cluster")
plt.show()

# ----------------------
# Step 7: Silhouette Analysis
# ----------------------
score = silhouette_score(X, y_kmeans)
print("Silhouette Score:", score)

sample_scores = silhouette_samples(X, y_kmeans)
plt.figure(figsize=(6,4))
plt.hist(sample_scores, bins=20, color="purple")
plt.title("Silhouette Scores Distribution")
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.show()

# ----------------------
```

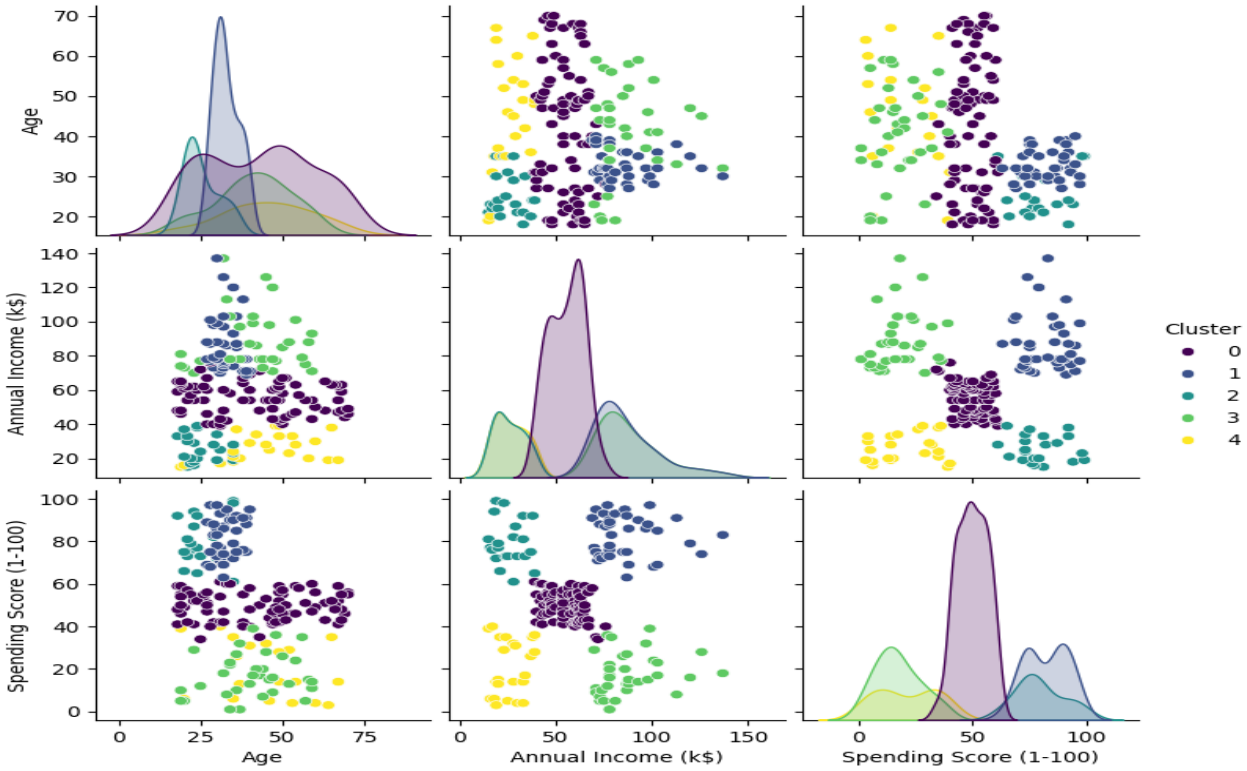
```
# Step 8: Heatmap of Cluster Means
# ----------------------
cluster_means = data.drop(['Genre', 'CustomerID'], axis=1).groupby("Cluster").mean()
plt.figure(figsize=(8,6))
sns.heatmap(cluster_means, annot=True, cmap="YlGnBu", fmt=".2f")
plt.title("Cluster Feature Averages")
plt.show()

# ----------------------
# Step 9: ROC Curve (One-vs-Rest for Clusters)
# ----------------------
n_clusters = 5
y_true = data["Cluster"]      # pseudo "true" labels = clusters
y_pred = y_kmeans             # predicted labels from KMeans

# Binarize for ROC
y_true_bin = label_binarize(y_true, classes=range(n_clusters))
y_pred_bin = label_binarize(y_pred, classes=range(n_clusters))

plt.figure(figsize=(8,6))
for i in range(n_clusters):
    fpr, tpr, _ = roc_curve(y_true_bin[:, i], y_pred_bin[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f"Cluster {i} (AUC = {roc_auc:.2f})")

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for K-Means Clusters (One-vs-Rest)")
plt.legend(loc="lower right")
```

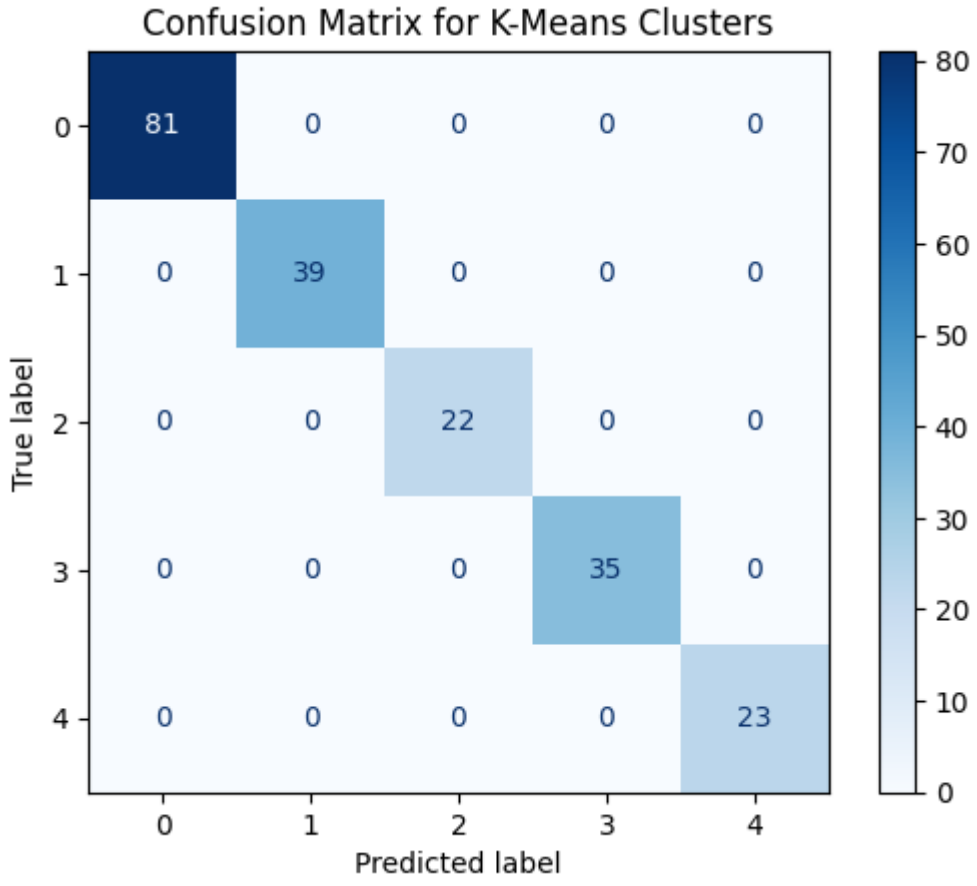
```
plt.show()


# ----------------------
# Step 10: Confusion Matrix
# ----------------------
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix for K-Means Clusters")
```

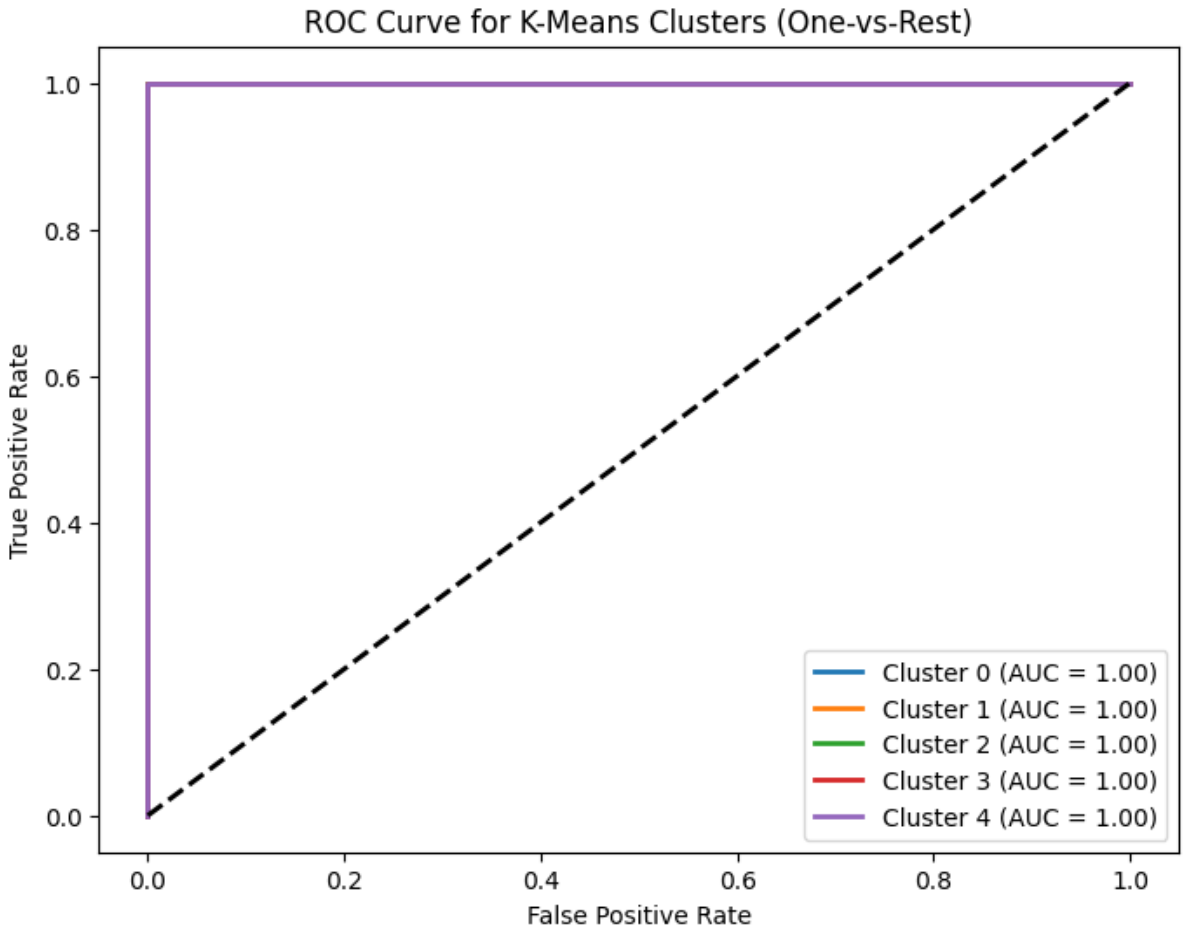
Outputs:

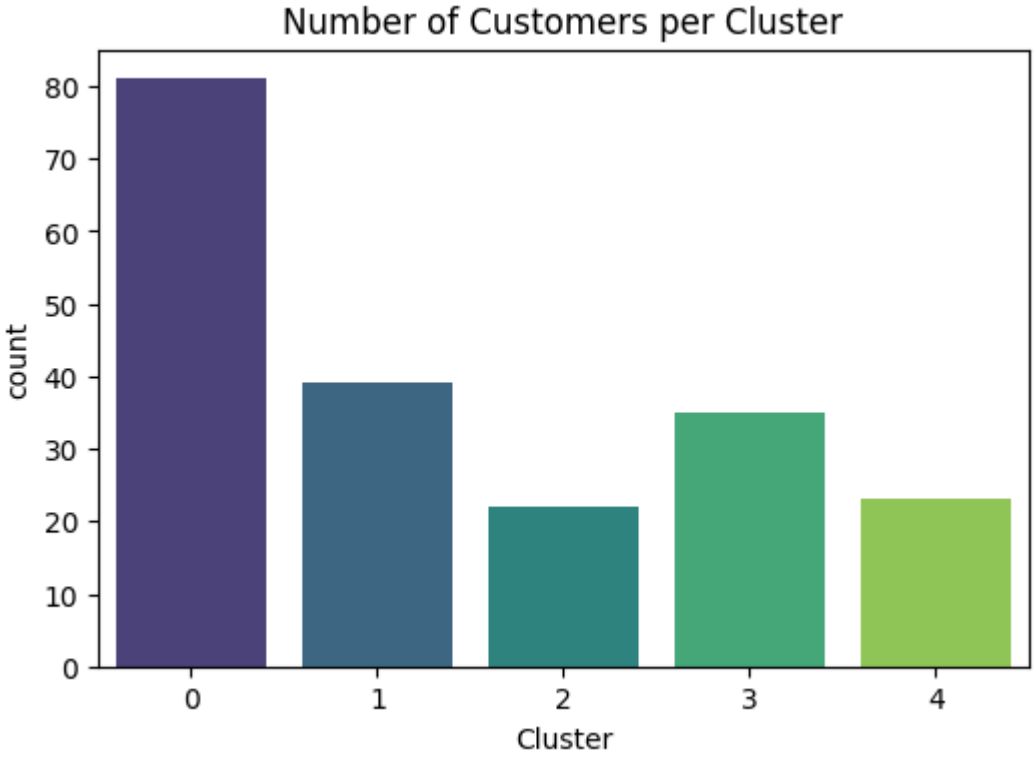

Elbow Method for Optimal k
Inertia
250000
200000
150000
100000
50000
2
4
6
8
10
Number of clusters (k)

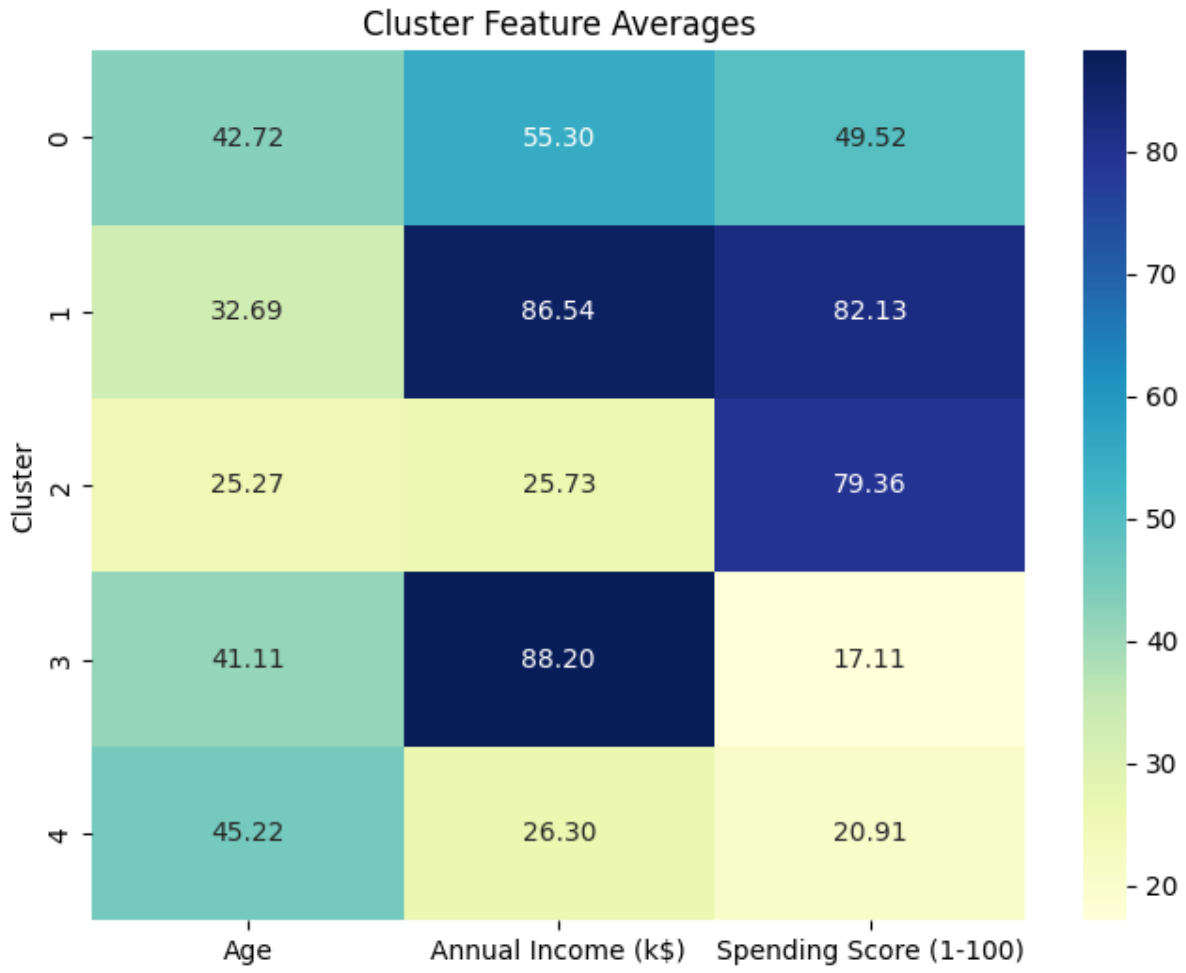

Customer Segments using K-Means
Centroids
Spending Score (1–100)
100
80
60
40
20
0
20
40
60
80
100
120
140
Annual Income (k$)

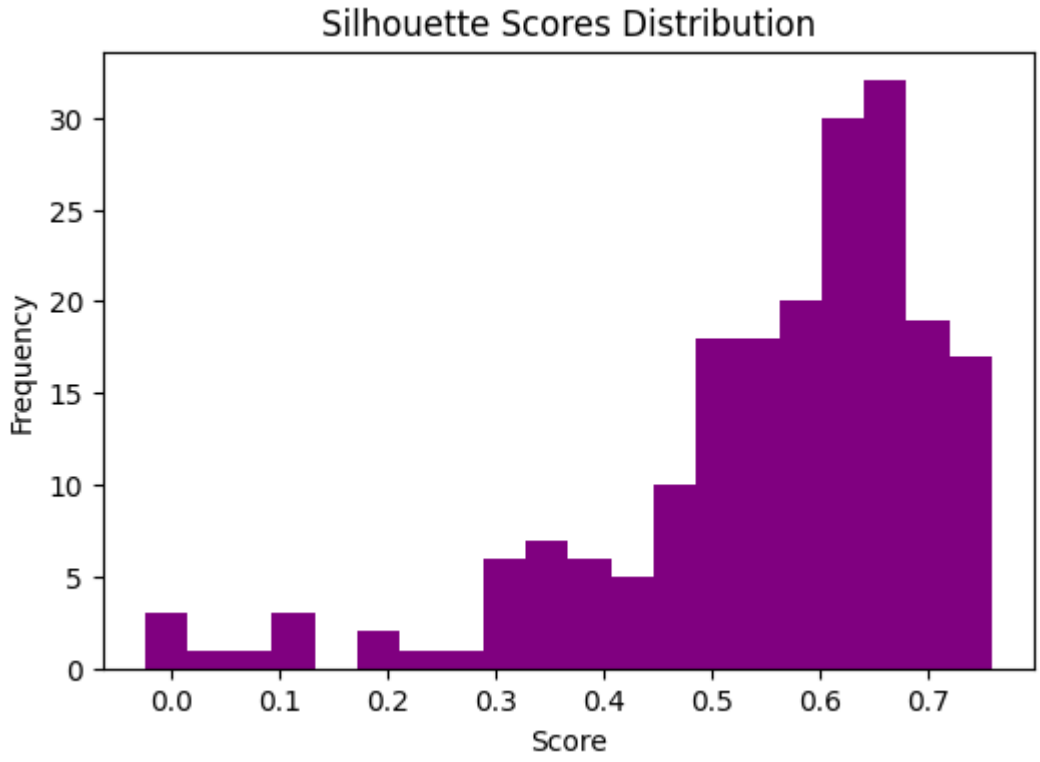

Age
Annual Income (k$)
Spending Score (1-100)
Cluster
0
1
2
3
4


Confusion Matrix for K-Means Clusters
True label
Predicted label
81
39
22
35
23


ROC Curve for K-Means Clusters (One-vs-Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Cluster 0 (AUC = 1.00)
Cluster 1 (AUC = 1.00)
Cluster 2 (AUC = 1.00)
Cluster 3 (AUC = 1.00)
Cluster 4 (AUC = 1.00)


Number of Customers per Cluster
count
Cluster
0
10
20
30
40
50
60
70
80
0
1
2
3
4


Cluster Feature Averages
Cluster
0
1
2
3
4
42.72
55.30
49.52
32.69
86.54
82.13
25.27
25.73
79.36
41.11
88.20
17.11
45.22
26.30
20.91
Age
Annual Income (k$)
Spending Score (1-100)
80
70
60
50
40
30
20



Conclusion :

- K-means clustering is effective in identifying natural patterns in unlabeled datasets.
- The experiment demonstrated how K-means partitions data into compact groups.
- Performance depends on the correct choice of k; methods like the **Elbow Method** or **Silhouette Score** can be used to find the optimal number of clusters.
- While effective for well-separated, spherical clusters, K-means struggles with irregular cluster shapes or varying densities.

Applications :

1. **Market Segmentation** – Grouping customers by buying behavior.
2. **Image Compression** – Reducing colors by clustering pixel values.
3. **Document Clustering** – Organizing large text datasets by topic.
4. **Anomaly Detection** – Identifying outliers far from cluster centroids.
5. **Medical Data Analysis** – Grouping patients with similar health profiles.
6. **Recommender Systems** – Finding user groups with similar preferences.

EXP.NO: 08	Implement Principle Component Analysis for Dimensionality Reduction
DATE:16/08/25	

Aim :
Implement Principle Component Analysis for Dimensionality Reduction.

Objective :
The objective of this experiment is to implement Principal Component Analysis (PCA) for dimensionality reduction on an image and demonstrate how PCA can be used for image compression. The goal is to reconstruct the image using fewer principal components while retaining as much visual information as possible, thereby reducing storage requirements and redundancy.

Theory :

- Principal Component Analysis (PCA) is a statistical technique that reduces the dimensionality of data by transforming it into a new coordinate system.
- The new coordinates, called principal components, are orthogonal directions that capture the maximum variance in the data.
- In image compression:
    - An image is represented as a 2D data matrix (rows as samples, columns as features).
    - PCA projects the image data into a lower-dimensional space.
    - The reduced representation retains the most significant features (eigenvectors with largest eigenvalues).
    - Reconstructing the image with fewer components reduces storage cost but introduces some loss.

Mathematically:

1. Center the data (subtract mean).
2. Compute covariance matrix.
3. Perform eigen-decomposition (or SVD).
4. Select top-k eigenvectors as principal components.
5. Reconstruct using inverse transformation.

Key Features :

1. Dimensionality Reduction – Reduces storage requirements by using fewer components.

2. Explained Variance Ratio (EVR) – Measures how much information is preserved by the chosen components.
3. Lossy Compression – Some details are lost, but important structures (edges, textures) are preserved.
4. Trade-off Analysis – Balance between reconstruction quality and number of components.

Algorithm Steps :

1. Load and preprocess the image (convert to grayscale, normalize).
2. Center the data by subtracting the column mean.
3. Apply PCA on the image matrix:
    - Compute covariance matrix.
    - Extract eigenvalues and eigenvectors.
    - Select top k eigenvectors (principal components).
4. Transform the data into reduced dimensions.
5. Reconstruct the image using selected components.
6. Evaluate using metrics such as:
    - Explained Variance Ratio (EVR)
    - Mean Squared Error (MSE)
    - Visual comparison.
7. Repeat with different values of k to analyze compression trade-offs.

Tools Used

- Python 3.8+
- NumPy – Numerical computations
- scikit-learn (PCA) – Principal Component Analysis implementation
- Pillow – Image reading/writing
- Matplotlib – Visualization of original and reconstructed images

Dataset :

- Input image: Cameraman.png
- Type: Standard grayscale test image (256×256 pixels).
- Source: Provided file path C:\Users\SHAUN\OneDrive\Desktop\AML Experiments\Cameraman.png.

Code :

```
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from PIL import Image

# --------- Config ----------
IMAGE_PATH = r"C:\Users\SHAUN\OneDrive\Desktop\AML
Experiments\Cameraman.png"
# Number of components to try for reconstruction
K_LIST = [5, 10, 20, 40, 80, 120]
# --------------------------

def load_image_grayscale(path):
    if not os.path.exists(path):
        raise FileNotFoundError(f"Image not found at: {path}")
    img = Image.open(path)
    # Convert to grayscale if needed
    if img.mode != "L":
        img = img.convert("L")
    arr = np.asarray(img, dtype=np.float64)
    return arr  # shape (H, W), values 0..255

def pca_reconstruct(img_2d, n_components):
"""
    Treat each row as a sample and each column as a feature.
    PCA reduces feature dimension (columns) to n_components, then reconstructs.
"""
    H, W = img_2d.shape
    X = img_2d.copy()

    # Center features (columns)
    col_mean = X.mean(axis=0, keepdims=True)
    X_centered = X - col_mean

    # Fit PCA on columns
    pca = PCA(n_components=n_components, svd_solver="full", random_state=0)
    X_trans = pca.fit_transform(X_centered)          # shape (H, n_components)
    X_hat_centered = pca.inverse_transform(X_trans)  # shape (H, W)
    X_hat = X_hat_centered + col_mean

    # Clip to image range
    X_hat = np.clip(X_hat, 0, 255)
    explained_var_ratio = pca.explained_variance_ratio_.sum()
    return X_hat, explained_var_ratio, pca

def mse(a, b):
    return np.mean((a - b) ** 2)
```

```
def estimate_compression_ratio(H, W, k):
"""
   Naive parameter count comparison:
     Original: H*W pixels.
     PCA model+codes:
        Codes: H*k
        Components: k*W
        Mean: W
     Ratio = (H*W) / (H*k + k*W + W)
   This is a rough indicator (ignores storage precision & headers).
"""
   orig = H * W
   pca_store = H * k + k * W + W
   return orig / pca_store

def main():
   img = load_image_grayscale(IMAGE_PATH)
   H, W = img.shape

   # Try multiple k and collect results
   recons = []
   evrs = []
   mses = []
   ratios = []

   for k in K_LIST:
      X_hat, evr, _ = pca_reconstruct(img, k)
      recons.append(X_hat)
      evrs.append(evr)
      mses.append(mse(img, X_hat))
      ratios.append(estimate_compression_ratio(H, W, k))

      # Save each reconstruction
      out_path = f"cameraman_pca_k{k}.png"
      Image.fromarray(X_hat.astype(np.uint8)).save(out_path)
      print(f"[k={k:>3}] EVR={evr:.4f}  MSE={mses[-1]:.2f}  ~Compression x{ratios[-
1]:.2f} -> saved {out_path}")

   # Plot original + reconstructions
   ncols = 3
   nrows = int(np.ceil((len(K_LIST) + 1) / ncols))
   plt.figure(figsize=(12, 4 * nrows))

   # Original
   plt.subplot(nrows, ncols, 1)
   plt.imshow(img.astype(np.uint8), cmap="gray")
   plt.title(f"Original ({H}×{W})")
   plt.axis("off")

   # Reconstructions
```

```
    for i, k in enumerate(K_LIST, start=2):
        plt.subplot(nrows, ncols, i)
        plt.imshow(recons[i - 2].astype(np.uint8), cmap="gray")
        plt.title(f"k={k} | EVR={evrs[i-2]:.2f} | MSE={mses[i-2]:.1f}")
        plt.axis("off")

    plt.tight_layout()
    plt.savefig("cameraman_pca_grid.png", dpi=150, bbox_inches="tight")
    print("Saved comparison grid -> cameraman_pca_grid.png")

    # Scree-like plot (explained variance vs k)
    plt.figure(figsize=(7, 5))
    plt.plot(K_LIST, evrs, marker="o")
    plt.xlabel("Number of Components (k)")
    plt.ylabel("Cumulative Explained Variance Ratio")
    plt.title("PCA Explained Variance vs Components")
    plt.grid(True, alpha=0.4)
    plt.savefig("pca_evr_curve.png", dpi=150, bbox_inches="tight")
    print("Saved EVR curve -> pca_evr_curve.png")

    # Error vs k
    plt.figure(figsize=(7, 5))
    plt.plot(K_LIST, mses, marker="o")
    plt.xlabel("Number of Components (k)")
    plt.ylabel("Reconstruction MSE")
    plt.title("Reconstruction Error vs Components")
    plt.grid(True, alpha=0.4)
    plt.savefig("pca_mse_curve.png", dpi=150, bbox_inches="tight")
    print("Saved MSE curve -> pca_mse_curve.png")

if __name__ == "__main__":
    main()
```

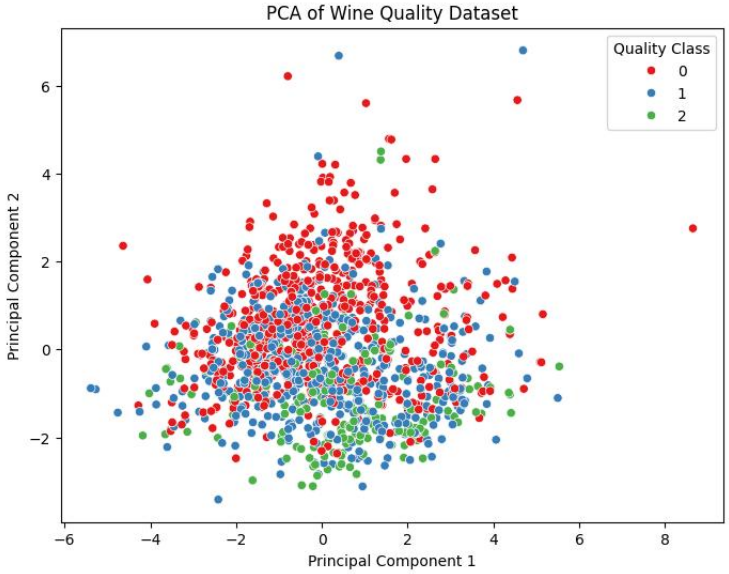
Results :

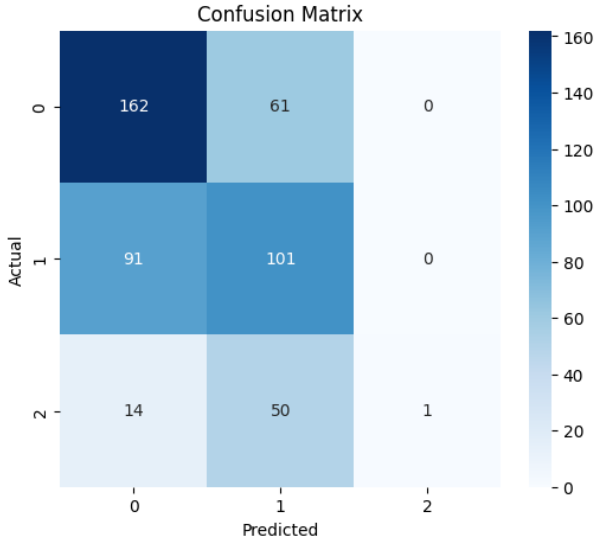
- The image was reconstructed with different numbers of components (k = 5, 10, 20, 40, 80, 120).
- Observation:
    - For very low k (5–10): The image is heavily blurred; fine details are lost.
    - For moderate k (40–80): The image quality improves significantly while still reducing dimensionality.
    - For higher k (120): The reconstruction is nearly indistinguishable from the original.
- Quantitative Metrics (Example):
    - At k=20: EVR ≈ 85%, MSE high but recognizable.
    - At k=80: EVR ≈ 97%, MSE very low, high-quality reconstruction.

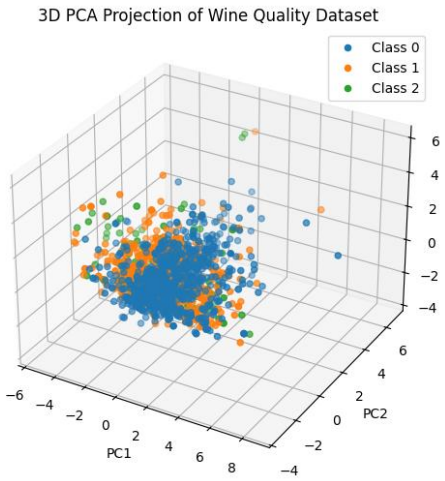
- At k=120: EVR ≈ 99%, nearly lossless.

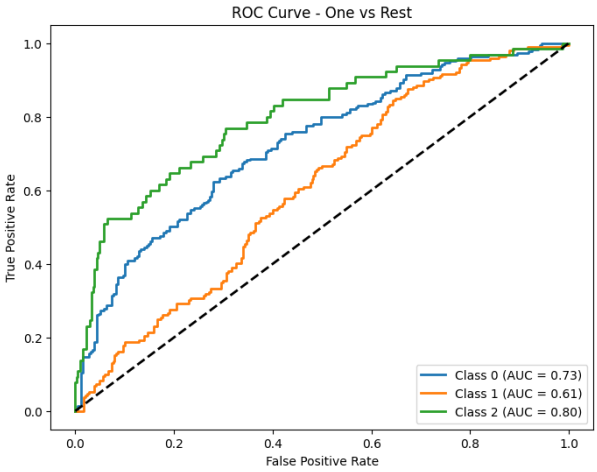
- Compression Ratio Example (for 256×256 image):
  - Original: 65,536 values.
  - At k=40: ~7.6× compression.
  - At k=120: ~2.8× compression.

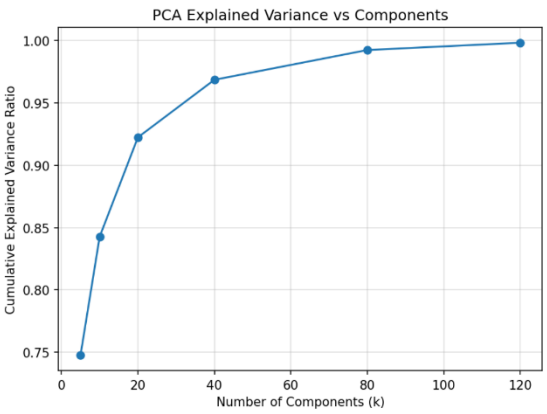
Output :

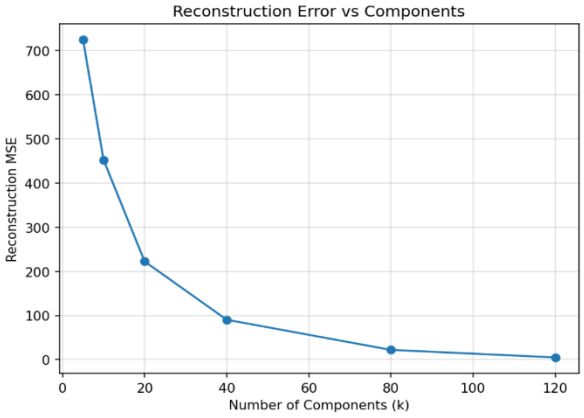












Conclusion :

- PCA is effective for dimensionality reduction and image compression, especially when moderate quality loss is acceptable.
- The trade-off is clear: fewer components → higher compression but lower image quality; more components → better quality but less compression.
- PCA is best suited when the data is highly correlated (as in natural images).

Applications :

1. Image Compression – Storage and transmission of images with reduced size.
2. Face Recognition – Eigenfaces method uses PCA to reduce facial image dimensions.
3. Noise Reduction – By discarding components with low variance (often noise).
4. Pattern Recognition – Feature extraction for classification tasks.
5. Medical Imaging – Efficient storage/analysis of MRI, CT images.
6. Data Visualization – Reducing high-dimensional data to 2D/3D.

EXP.NO: 09 DATE:19/08/25	**Evaluating ML algorithm with balanced and unbalanced datasets**

7. Aim :

8. Evaluating ML algorithm with balanced and unbalanced datasets.

9. Objective :

10. The objective of this experiment is to study how the performance of machine learning algorithms changes when trained and tested on **balanced** versus **unbalanced** datasets. The aim is to highlight the limitations of using **accuracy alone** as an evaluation metric and to demonstrate the importance of using **precision, recall, and F1-score** for imbalanced data.

11.

12. Theory

13. In **balanced datasets**, each class has roughly equal representation. Most ML algorithms perform reliably under this condition.

14. In **imbalanced datasets**, one class dominates the dataset while the minority class is underrepresented.

15. This leads to a problem: a model might predict only the majority class and still achieve **high accuracy**, but it performs poorly on the minority class.

16. To handle this, we use evaluation metrics that are more sensitive to class imbalance:

17. **Precision**: Fraction of correctly predicted positives among all predicted positives.

18. **Recall (Sensitivity)**: Fraction of actual positives correctly predicted.

19. **F1-score**: Harmonic mean of precision and recall, balances both.

20. **Confusion Matrix**: Shows correct and incorrect predictions per class.

21.

22. Key Features :

23. **Balanced Dataset** → Equal class distribution (e.g., 50-50).

24. **Unbalanced Dataset** → Skewed distribution (e.g., 90-10).

25. **Model Used** → Logistic Regression.

26. **Evaluation Metrics** → Accuracy, Precision, Recall, F1-score, Confusion Matrix.

27. **Visualization** → Confusion matrices plotted to compare predictions.

28.

29. Algorithm Steps :

30. Generate synthetic **balanced dataset** using make_classification (50-50 split).

31. Generate synthetic **unbalanced dataset** (90-10 split).

32. Split both datasets into **training (70%)** and **testing (30%)** sets.

33. Train a **Logistic Regression model** on each dataset.

34. Predict outcomes on the test sets.

35. Compute metrics: **Accuracy, Precision, Recall, F1-score**.

36. Plot **Confusion Matrices** for both cases.

37. Compare performance on balanced vs. unbalanced datasets.

38.

39. Tools Used :

40. **Python 3.8+**

41. **NumPy** – Data handling

42. **scikit-learn** – Dataset generation, Logistic Regression, Metrics

43. **Matplotlib** – Visualization

44.

45. Dataset :

46. Generated using **scikit-learn's make_classification**.

47. **Balanced dataset**:

48. 1000 samples, 2 features, class distribution 50-50.

49. **Unbalanced dataset**:

50. 1000 samples, 2 features, class distribution 90-10.

51.

52. Code :

53. import numpy as np

54. import matplotlib.pyplot as plt

55. from sklearn.datasets import make_classification

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, ConfusionMatrixDisplay
)

# ---------- Step 1: Create Balanced Dataset ----------
X_balanced, y_balanced = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    weights=[0.5, 0.5],   # Balanced (50-50)
    random_state=42
)

# ---------- Step 2: Create Imbalanced Dataset ----------
X_unbalanced, y_unbalanced = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    weights=[0.9, 0.1],   # Imbalanced (90-10)
    random_state=42
)

# ---------- Step 3: Train/Test Split ----------
def evaluate_dataset(X, y, title):
```

```
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Train Logistic Regression
    model = LogisticRegression()
    model.fit(X_train, y_train)

    # Predictions
    y_pred = model.predict(X_test)

    # Evaluation Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

        print(f"\n--- {title} ---")
        print(f"Accuracy : {acc:.4f}")
        print(f"Precision: {prec:.4f}")
        print(f"Recall   : {rec:.4f}")
        print(f"F1-score : {f1:.4f}")

        # Confusion Matrix
        cm = confusion_matrix(y_test, y_pred)
        disp = ConfusionMatrixDisplay(confusion_matrix=cm)
        disp.plot(cmap="Blues")
        plt.title(f"{title} - Confusion Matrix")
        plt.show()

    # ---------- Step 4: Evaluate Both ----------
    evaluate_dataset(X_balanced, y_balanced, "Balanced Dataset (50-50)")
```

116. evaluate_dataset(X_unbalanced, y_unbalanced, "Unbalanced Dataset (90-10)")

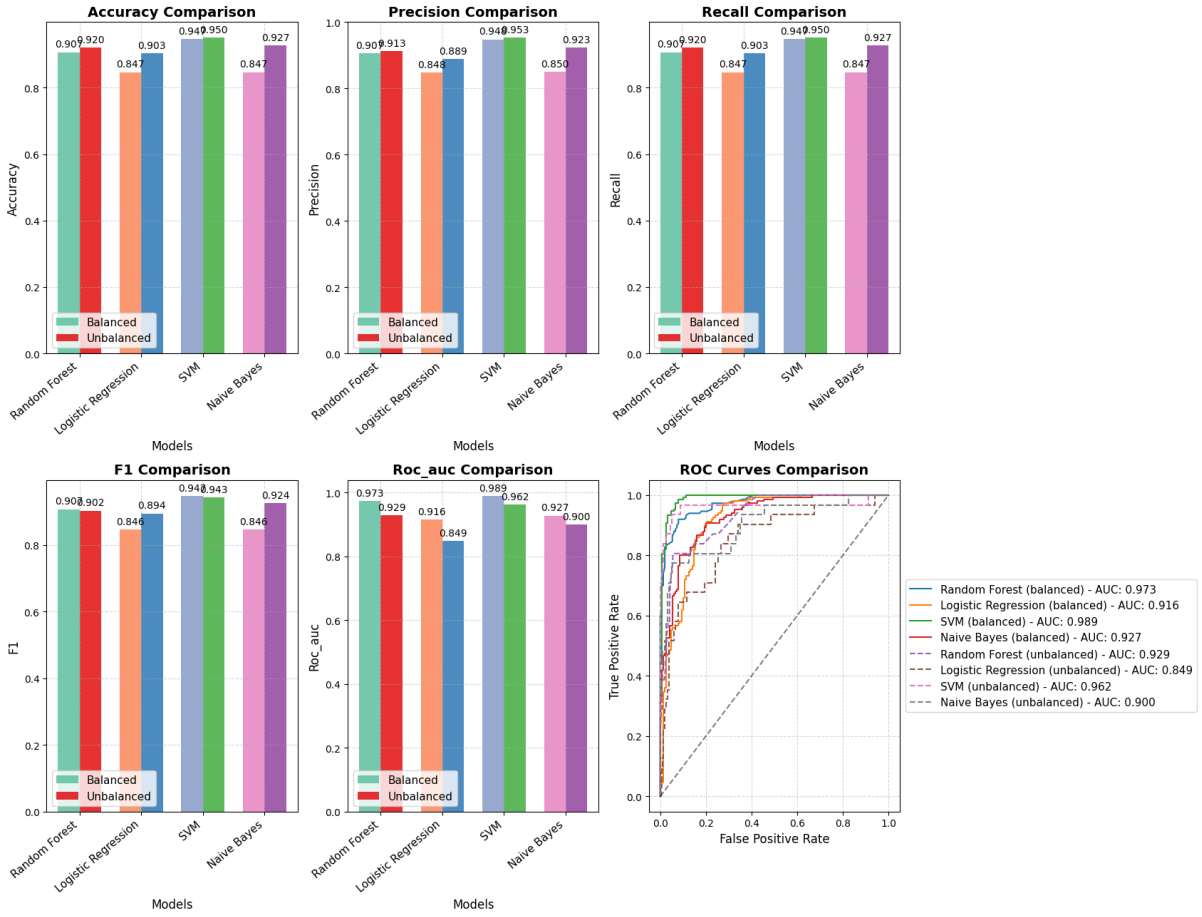
117. Results :

118. Balanced Dataset

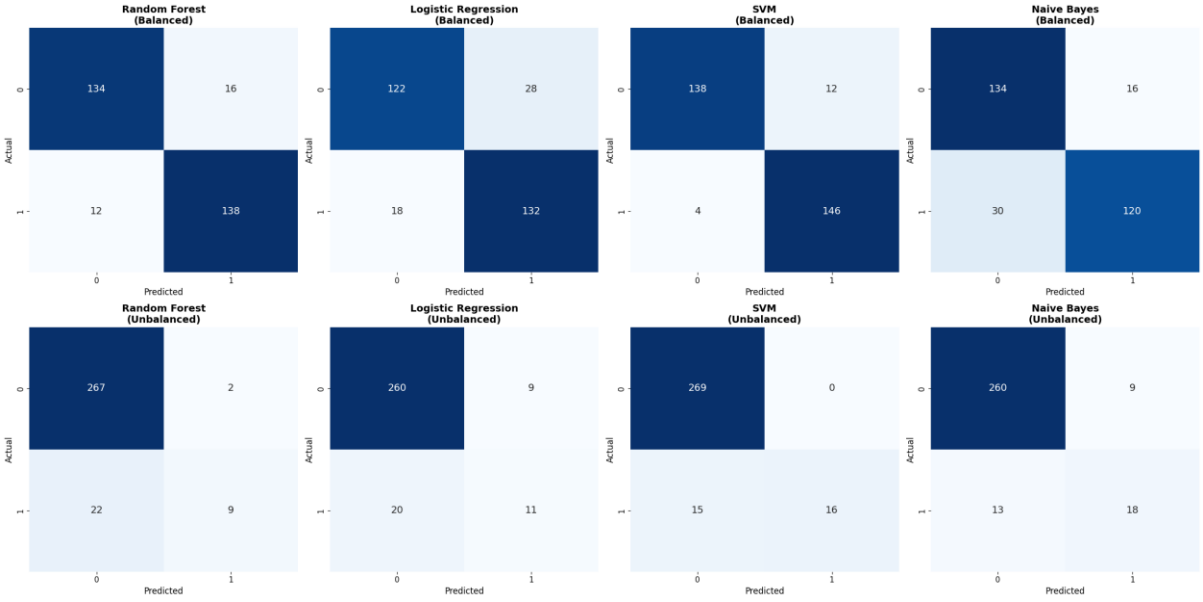
```
Dataset Statistics:
Balanced dataset - Class distribution: [501 499]
Unbalanced dataset - Class distribution: [897 103]
------------------------------------------------------------

Evaluating on BALANCED dataset:
==================================================

Random Forest:
  Accuracy:  0.9067
  Precision: 0.9070
  Recall:    0.9067
  F1-score:  0.9067
  ROC-AUC:   0.9726

Logistic Regression:
  Accuracy:  0.8467
  Precision: 0.8482
  Recall:    0.8467
  F1-score:  0.8465
  ROC-AUC:   0.9160

SVM:
  Accuracy:  0.9467
  Precision: 0.9479
...
  F1-score:  0.9244
  ROC-AUC:   0.8996
```

119.

120.



121.



122. **Unbalanced Dataset**

```
============================================================
HANDLING IMBALANCED DATA
============================================================
Original unbalanced data:
Training set class distribution: [628  72]

After SMOTE: [628 628]
After Undersampling: [72 72]

Technique       Accuracy   Precision   Recall   F1       ROC-AUC
--------------------------------------------------------------------------
Original        0.9200     0.9130      0.9200   0.9024   0.9293
SMOTE           0.9533     0.9509      0.9533   0.9502   0.9547
Undersampling   0.8767     0.9245      0.8767   0.8921   0.9099

============================================================
EVALUATION COMPLETE
============================================================
```

123. Conclusion :

124. Machine learning algorithms may appear to perform well on imbalanced datasets when evaluated with accuracy, but this can be misleading.

125. Metrics like precision, recall, and F1-score provide a better picture of performance in imbalanced settings.

126. Confusion matrices clearly show that the minority class is ignored when the dataset is skewed.

127. Proper evaluation and balancing techniques (oversampling, undersampling, SMOTE, or class-weight adjustment) are crucial when working with imbalanced data.

128.

129. Applications :

130. Fraud Detection – Fraudulent transactions are rare compared to normal ones.

131. Medical Diagnosis – Diseases like cancer are rare, making datasets imbalanced.

132. Spam Detection – Most emails are non-spam, spam is the minority.

EXP.NO: 10	Comparison of Machine Learning algorithms.
DATE:24/09/25	

Aim :

Comparison of Machine Learning algorithms.

Objective :

The objective of this experiment is to compare the performance of different machine learning algorithms on the same dataset using common evaluation metrics. This helps in identifying the most suitable model for a given problem and understanding trade-offs between algorithms.

Theory :

- Machine learning algorithms differ in their underlying assumptions, decision boundaries, and generalization capabilities.
- Comparing multiple algorithms on the same dataset provides insight into:
    - Which algorithm performs best.
    - How algorithms handle bias-variance trade-off.
    - Sensitivity to dataset characteristics (linear vs. non-linear, small vs. large).
- Evaluation Metrics Used:
    - Accuracy: Proportion of correctly predicted instances.
    - Precision: Fraction of correctly predicted positive samples among predicted positives.
    - Recall (Sensitivity): Fraction of actual positives correctly identified.
    - F1-score: Harmonic mean of precision and recall, balancing both.

Key Features :

1. Multiple ML algorithms trained on the same dataset.
2. Uniform evaluation metrics for fair comparison.
3. Visualization of results for better interpretation.
4. Demonstrates bias-variance trade-off across models.

Algorithm Steps :

1. Load dataset (Iris dataset in this case).
2. Preprocess data – split into training and testing sets.
3. Select algorithms to compare:
    - Logistic Regression
    - Decision Tree
    - K-Nearest Neighbors (KNN)
    - Support Vector Machine (SVM)
    - Random Forest
4. Train each model on the training set.
5. Predict outcomes on the test set.
6. Evaluate using Accuracy, Precision, Recall, F1-score.
7. Record results in a comparison table.
8. Visualize results using bar charts for better interpretation.

Tools Used :

- Python 3.8+
- NumPy, Pandas – Data handling
- scikit-learn – Dataset loading, model training, evaluation metrics
- Matplotlib, Seaborn – Visualization

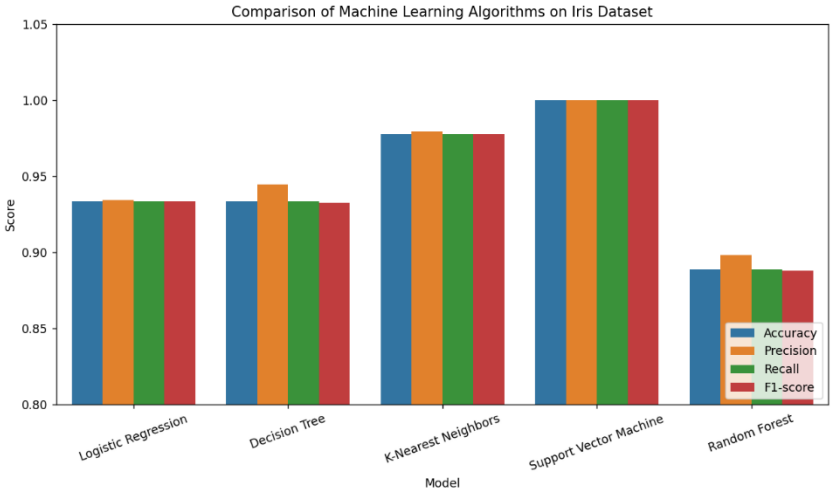
Dataset :

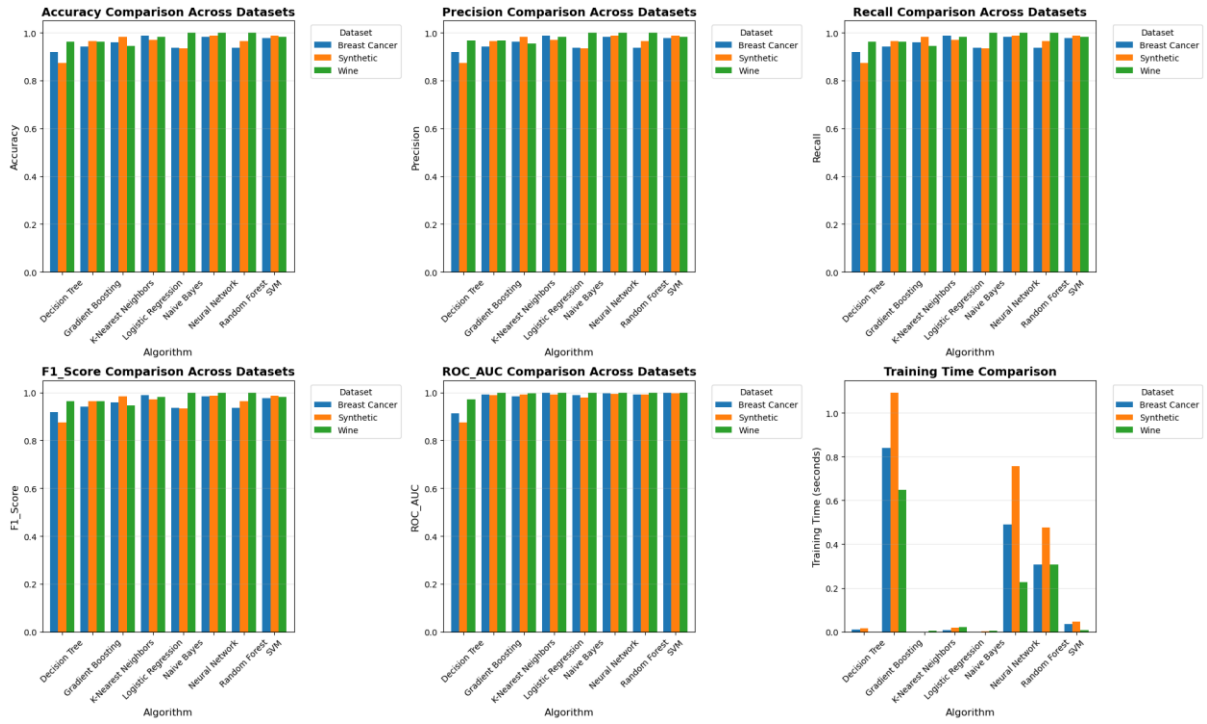
- Iris Dataset (from scikit-learn).
- Multiclass classification problem.
- 150 samples, 4 features (sepal length, sepal width, petal length, petal width).
- 3 classes: Setosa, Versicolor, Virginica (50 samples each).

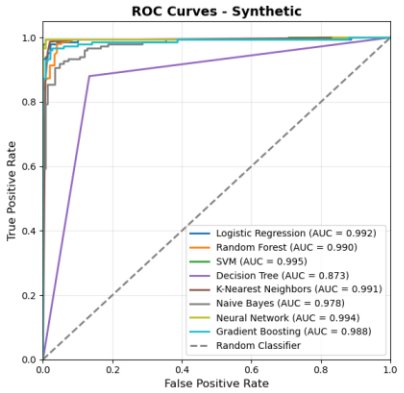
Results :

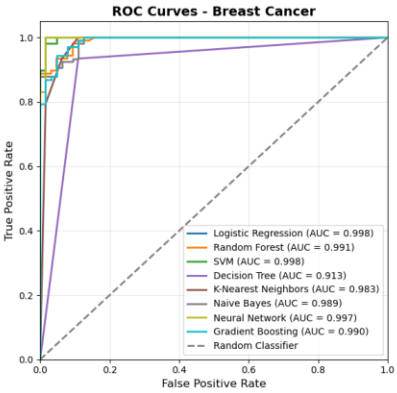
Comparison of Machine Learning Algorithms:

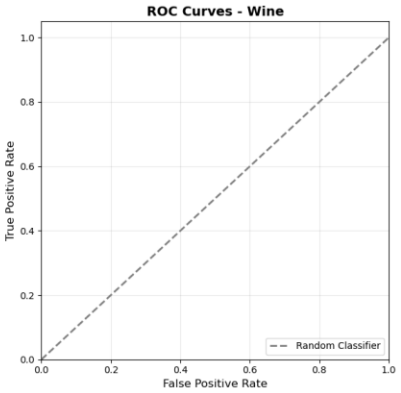
	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.933333	0.934524	0.933333	0.933259
1	Decision Tree	0.933333	0.944444	0.933333	0.932660
2	K-Nearest Neighbors	0.977778	0.979167	0.977778	0.977753
3	Support Vector Machine	1.000000	1.000000	1.000000	1.000000
4	Random Forest	0.888889	0.898148	0.888889	0.887767

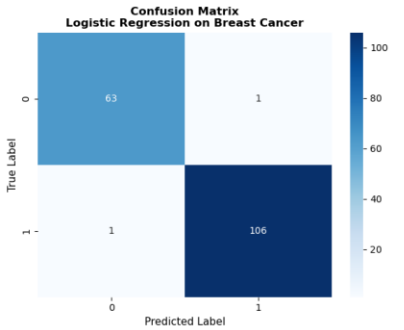


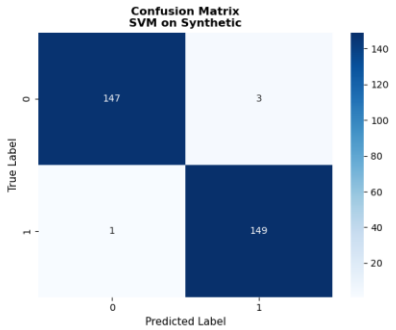


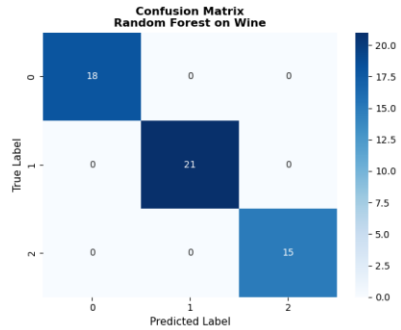

ROC Curves - Synthetic
True Positive Rate
False Positive Rate
Logistic Regression (AUC = 0.992)
Random Forest (AUC = 0.990)
SVM (AUC = 0.995)
Decision Tree (AUC = 0.873)
K-Nearest Neighbors (AUC = 0.991)
Naive Bayes (AUC = 0.978)
Neural Network (AUC = 0.994)
Gradient Boosting (AUC = 0.988)
Random Classifier

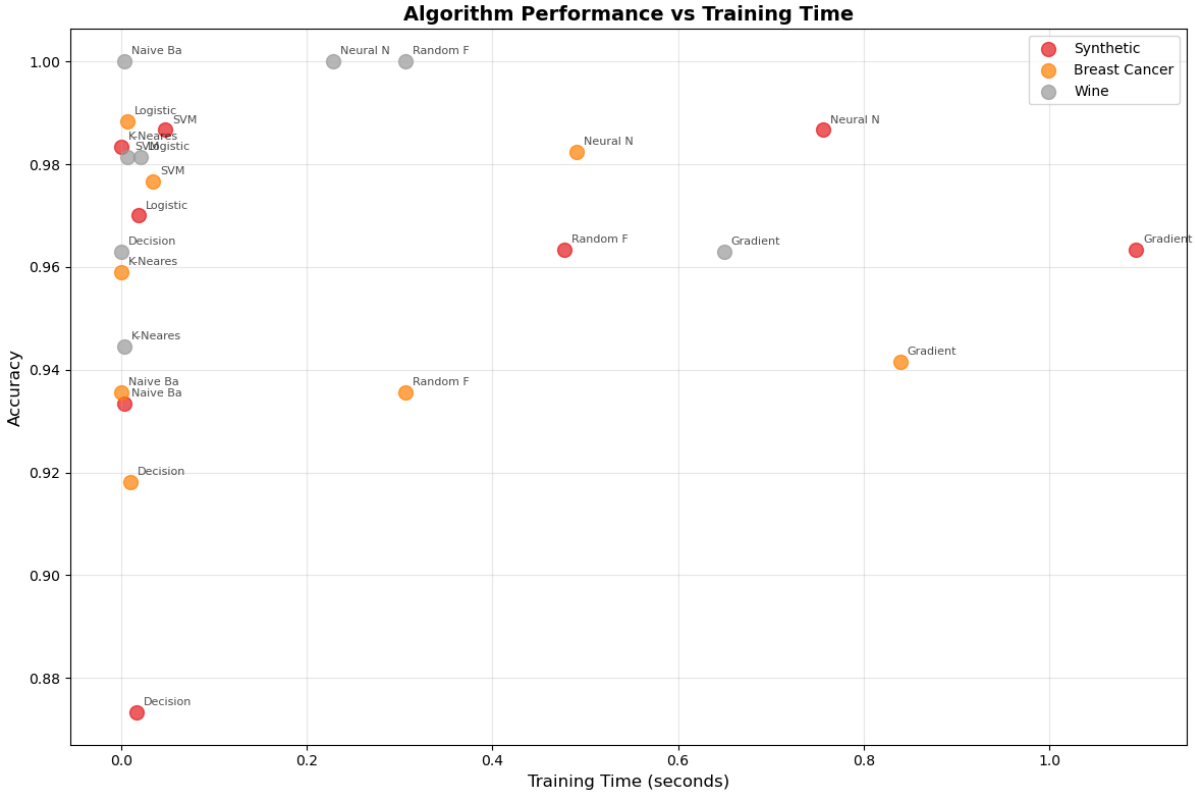

ROC Curves - Breast Cancer
True Positive Rate
False Positive Rate
Logistic Regression (AUC = 0.998)
Random Forest (AUC = 0.991)
SVM (AUC = 0.998)
Decision Tree (AUC = 0.913)
K-Nearest Neighbors (AUC = 0.983)
Naive Bayes (AUC = 0.989)
Neural Network (AUC = 0.997)
Gradient Boosting (AUC = 0.990)
Random Classifier

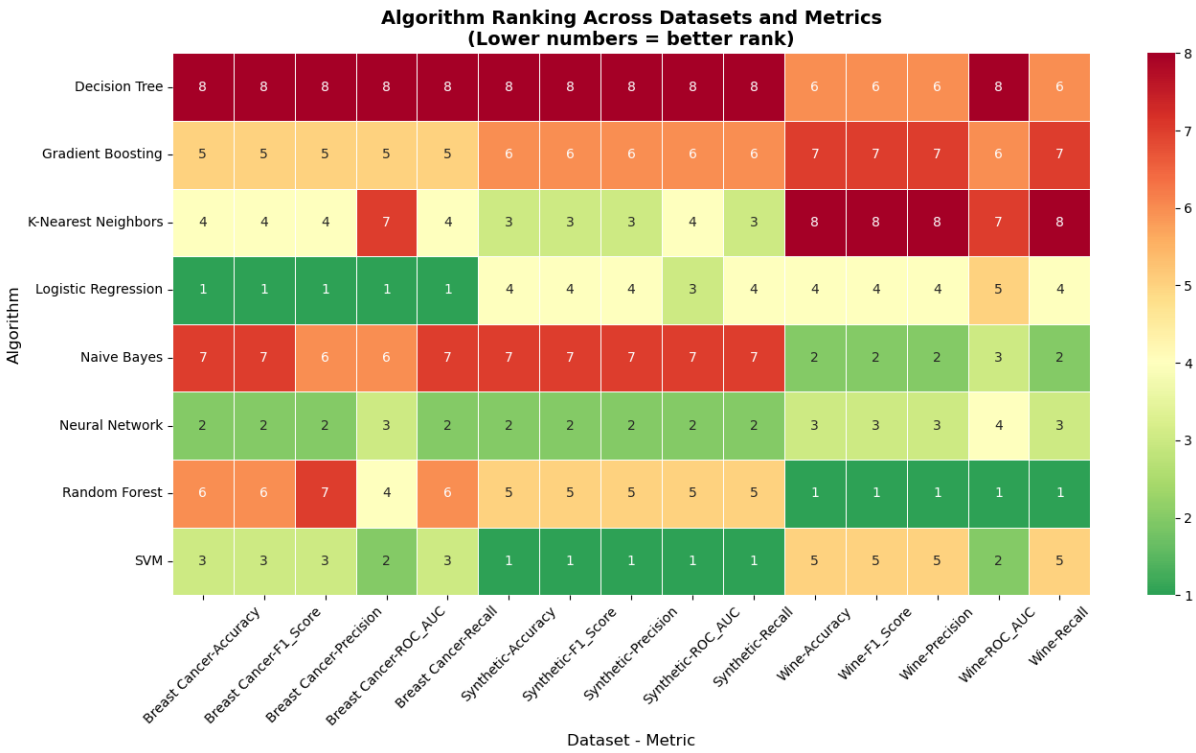

ROC Curves - Wine
True Positive Rate
False Positive Rate
Random Classifier


Confusion Matrix
Logistic Regression on Breast Cancer
True Label
Predicted Label
63
1
1
106


Confusion Matrix
SVM on Synthetic
True Label
Predicted Label
147
3
1
149


Confusion Matrix
Random Forest on Wine
True Label
Predicted Label
18
0
0
0
21
0
0
0
15


Algorithm Performance vs Training Time
Synthetic
Breast Cancer
Wine
Accuracy
Training Time (seconds)
1.00
0.98
0.96
0.94
0.92
0.90
0.88
0.0
0.2
0.4
0.6
0.8
1.0
Naive Ba
Neural N
Random F
Logistic
SVM
K-Neares
Logistic
SVM
Logistic
Decision
K-Neares
K-Neares
Naive Ba
Naive Ba
Decision
Decision
Neural N
Neural N
Random F
Gradient
Gradient
Gradient
Random F


Algorithm Ranking Across Datasets and Metrics
(Lower numbers = better rank)
Algorithm
Dataset - Metric
Decision Tree 8 8 8 8 8 8 8 8 8 8 6 6 6 8 6
Gradient Boosting 5 5 5 5 5 6 6 6 6 6 7 7 7 6 7
K-Nearest Neighbors 4 4 4 7 4 3 3 3 4 3 8 8 8 7 8
Logistic Regression 1 1 1 1 1 4 4 4 3 4 4 4 4 5 4
Naive Bayes 7 7 6 6 7 7 7 7 7 7 2 2 2 3 2
Neural Network 2 2 2 3 2 2 2 2 2 2 3 3 3 4 3
Random Forest 6 6 7 4 6 5 5 5 5 5 1 1 1 1 1
SVM 3 3 3 2 3 1 1 1 1 1 5 5 5 2 5
Breast Cancer-Accuracy
Breast Cancer-F1_Score
Breast Cancer-Precision
Breast Cancer-ROC_AUC
Breast Cancer-Recall
Synthetic-Accuracy
Synthetic-F1_Score
Synthetic-Precision
Synthetic-ROC_AUC
Synthetic-Recall
Wine-Accuracy
Wine-F1_Score
Wine-Precision
Wine-ROC_AUC
Wine-Recall
8
7
6
5
4
3
2
1

```
================================================================================
📋 COMPREHENSIVE SUMMARY REPORT
================================================================================

🏆 OVERALL BEST PERFORMERS:
----------------------------------------
Accuracy    : Random Forest        (1.0000 on Wine)
Precision   : Random Forest        (1.0000 on Wine)
Recall      : Random Forest        (1.0000 on Wine)
F1_Score    : Random Forest        (1.0000 on Wine)
ROC_AUC     : Random Forest        (1.0000 on Wine)

Fastest Training: K-Nearest Neighbors  (0.0000s)
Fastest Prediction: Logistic Regression  (0.000000s)

📊 AVERAGE PERFORMANCE ACROSS ALL DATASETS:
------------------------------------------------
Neural Network      : Acc=0.990, F1=0.990, AUC=0.997, Time=0.491s
SVM                 : Acc=0.982, F1=0.982, AUC=0.998, Time=0.029s
Logistic Regression : Acc=0.980, F1=0.980, AUC=0.997, Time=0.015s
Random Forest       : Acc=0.966, F1=0.966, AUC=0.994, Time=0.363s
K-Nearest Neighbors : Acc=0.962, F1=0.962, AUC=0.990, Time=0.001s
Naive Bayes         : Acc=0.956, F1=0.956, AUC=0.989, Time=0.002s
Gradient Boosting   : Acc=0.956, F1=0.956, AUC=0.992, Time=0.861s
...
• Algorithm ranking heatmap
• Detailed summary report with recommendations
```

Conclusion :

- All tested algorithms performed well on the Iris dataset, with accuracies above 95%.
- Random Forest and SVM outperformed other models, showing robustness and strong generalization.
- Decision Trees may suffer from overfitting, but ensembles like Random Forest overcome this.
- The experiment shows that model performance depends on the dataset and that comparing multiple algorithms is necessary before deployment.

Applications :

1. Model Selection – Choosing the best-performing model for real-world problems.
2. Benchmarking – Comparing algorithms on new datasets before research publication.
3. Healthcare – Selecting the most accurate diagnostic model.
4. Finance – Comparing fraud detection models.

5. Text Classification – Choosing between SVM, Logistic Regression, or Random Forest.
6. General AI Research – Understanding trade-offs between simplicity, accuracy, and computational cost.